# Foundations of Spatial AI

## Lecture 06 — Simultaneous Localization and Mapping

Detailed lecture notes

Notes on the slides of L. Di Giammarino and E. Giacomini
Sapienza University of Rome, October 12, 2026

## Contents

**6.1 The SLAM Problem *(slides 3–6)*** **1**
6.1.1 The SLAM Problem *(slide 3)* . . . 1
6.1.2 Cameras for SLAM *(slide 4)* . . . 2
6.1.3 SLAM and Structure-from-Motion *(slide 5)* . . . 3
6.1.4 Running Example: VBR Campus *(slide 6)* . . . 3

**6.2 Filtering: From the Bayes Filter to EKF-SLAM *(slides 7–13)*** **4**
6.2.1 SLAM as a Graphical Model *(slide 8)* . . . 4
6.2.2 The Bayes Filter *(slide 9)* . . . 5
6.2.3 Kalman Filter and Extended Kalman Filter *(slide 10)* . . . 5
6.2.4 The EKF-SLAM State *(slide 11)* . . . 6
6.2.5 Example: MonoSLAM *(slide 12)* . . . 7
6.2.6 Limits of Filtering *(slide 13)* . . . 9

**6.3 Graph-Based SLAM: Factor Graphs and Least Squares *(slides 14–22)*** **9**
6.3.1 Graph-Based SLAM *(slide 15)* . . . 9
6.3.2 Factor Graphs *(slide 16)* . . . 10
6.3.3 Visual SLAM as a Factor Graph *(slide 17)* . . . 11
6.3.4 The Pose Graph *(slide 18)* . . . 11
6.3.5 From Factors to Least Squares *(slide 19)* . . . 11
6.3.6 Error and Information Matrix of an Edge *(slide 20)* . . . 12
6.3.7 Gauss-Newton on the Pose Graph *(slide 21)* . . . 12
6.3.8 Sparsity of the Normal Equations *(slide 22)* . . . 13

**6.4 The Visual SLAM Pipeline: ORB-SLAM *(slides 23–32)*** **14**
6.4.1 Pipeline Overview *(slide 24)* . . . 14
6.4.2 ORB Features and Matching *(slide 25)* . . . 15
6.4.3 Map Initialization *(slide 26)* . . . 15
6.4.4 Tracking *(slide 27)* . . . 17
6.4.5 Keyframes *(slide 28)* . . . 18
6.4.6 Local Mapping *(slide 29)* . . . 19
6.4.7 Loop Detection with Bags of Binary Words *(slide 30)* . . . 21
6.4.8 Loop Correction and Scale Drift *(slide 31)* . . . 22
6.4.9 Relocalization *(slide 32)* . . . 24

**6.5 Research Directions *(slides 33–40)*** **25**
6.5.1 Continuous Time: Rolling-Shutter Cameras *(slide 34)* . . . 25
6.5.2 Continuous Time: Cumulative B-Splines on SE(3) *(slide 35)* . . . 26
6.5.3 Active SLAM *(slide 36)* . . . 27
6.5.4 Active SLAM with an Active Stereo Head *(slide 37)* . . . 28
6.5.5 Active Localization: Learning Where to Look *(slide 38)* . . . 29
6.5.6 Active View Selection with Radiance Fields *(slide 39)* . . . 29
6.5.7 Outlook: Learned Front Ends *(slide 40)* . . . 30

**6.6 Summary *(slide 41)*** **31**

## About these notes

The notes follow the slide deck in order; every `\section` and `\subsection` carries the title of the corresponding slide(s) and its slide range. Formulas on the slides are embedded as pictures, so the formal content (definitions, lemmas, theorems, proofs) was reconstructed from the references quoted in the slide footers (Thrun–Burgard–Fox, Stachniss, Dellaert–Kaess, Mur-Artal et al., Lovegrove et al. and others) and from lectures 01 and 05 (Lie groups, bundle adjustment). Dashed empty boxes mark photographs or screenshots that must be copied from the indicated slide; all other figures are redrawn in TikZ/pgfplots.

**Notation.** World-to-camera poses $\mathbf{T}_i \in \mathrm{SE}(3)$ (as in lecture 05); 3D points $\mathbf{X}_j$ (or $\mathbf{x}_j^w$); pixel observations $\mathbf{u}_{ij}$; $\pi(\cdot)$ is the pinhole projection including intrinsics; $\|\mathbf{v}\|_\Omega^2 = \mathbf{v}^\top \Omega\, \mathbf{v}$ is the squared Mahalanobis norm. The letter $\mathbf{H}$ is the Gauss–Newton matrix of Section 6.3, while the EKF measurement Jacobian is called $\mathbf{C}_t$.

## Agenda and Learning Objectives *(slides 1–2)*

**Agenda:** 6.1 The SLAM problem; 6.2 Filtering; 6.3 Graph-based SLAM; 6.4 The visual SLAM pipeline; 6.5 Research directions.

After the lecture you should be able to:

1. formulate SLAM as joint estimation of trajectory and map, and derive the Kalman filter and EKF-SLAM together with their cost;
2. formulate graph-based SLAM as least squares over a factor graph, solve it with Gauss–Newton, and relate its sparsity to marginalization;
3. describe the threads of a keyframe-based visual SLAM system: tracking, local mapping, loop detection and loop correction.

## 6.1 The SLAM Problem *(slides 3–6)*

### 6.1.1 The SLAM Problem *(slide 3)*

**Definition 6.1.1: SLAM**

*Simultaneous localization and mapping* computes, at the same time, the camera poses (the trajectory) $\mathbf{x}_{1:t} = \{\mathbf{x}_1, \dots, \mathbf{x}_t\}$ and the map of the environment $\mathbf{m}$, given the measurements $\mathbf{z}_{1:t}$ (and, if available, controls $\mathbf{u}_{1:t}$), i.e. the posterior

$$p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}).$$

*Localization* estimates the poses given the map; *mapping* builds the map given the poses.

SLAM is a *chicken-or-egg problem*: a map is needed to localize, and a pose estimate is needed to build a map. Without any map, chaining relative motion estimates (*dead reckoning*) accumulates error without bound.

**Proposition 6.1.2: Dead-reckoning drift**

If each relative motion estimate carries independent zero-mean error of variance $\sigma^2$ per axis, the position variance after $t$ steps is $t\sigma^2$, so the standard deviation grows as $\sigma\sqrt{t}$ and is unbounded.

*Proof.* The estimated position is $\sum_{k=1}^{t}(\Delta_k + \varepsilon_k)$ with independent $\varepsilon_k$; variances of independent terms add. □

**Intuition**

With $\sigma = 1\,\text{cm}$ per step, after 1000 steps the $1\sigma$ error is $\sqrt{1000} \approx 32\,\text{cm}$, even though no single step was bad. Orientation errors are worse: a small heading error bends the whole subsequent path. A map that is *re-observed* gives absolute references and stops this growth.

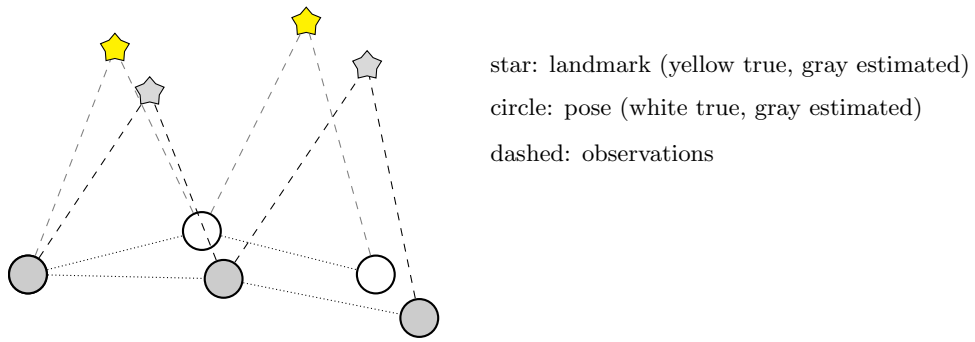



Figure 1: Redrawn from slide 3: true and estimated poses/landmarks and the observations linking them. Estimated poses drift away from the truth while the landmarks keep them mutually consistent.

### 6.1.2 Cameras for SLAM *(slide 4)*

Cameras are compact, accurate, non-invasive, well understood, and today cheap and ubiquitous. Images carry *appearance*: two rooms that look identical to a 2D laser scanner can be told apart by a camera. Applications range from augmented reality and vision-based navigation to self-driving cars, mobile devices, inspection drones and VR headsets. The cost: a camera senses geometry only indirectly, through photometric effects, and a single camera measures *no depth*.

**Proposition 6.1.3: Scale ambiguity of a monocular camera**

Let a scene $\{\mathbf{X}_j\}$ and camera poses $\{(\mathbf{R}_i, \mathbf{t}_i)\}$ produce pixels $\mathbf{u}_{ij} = \pi(\mathbf{R}_i\mathbf{X}_j + \mathbf{t}_i)$. For any $s > 0$, the scene $\{s\mathbf{X}_j\}$ with translations $\{s\mathbf{t}_i\}$ produces exactly the same pixels.

*Proof.* $\mathbf{R}_i(s\mathbf{X}_j) + s\mathbf{t}_i = s(\mathbf{R}_i\mathbf{X}_j + \mathbf{t}_i)$, and the perspective projection satisfies $\pi(s\mathbf{p}) = \pi(\mathbf{p})$ because it divides by the depth. □

**Intuition**

A photograph of a toy house and of a real house can be identical. A single camera therefore recovers the world only *up to a similarity* (rotation, translation *and* scale: 7 degrees of

freedom), which is why monocular maps also drift in scale (Section 6.4).

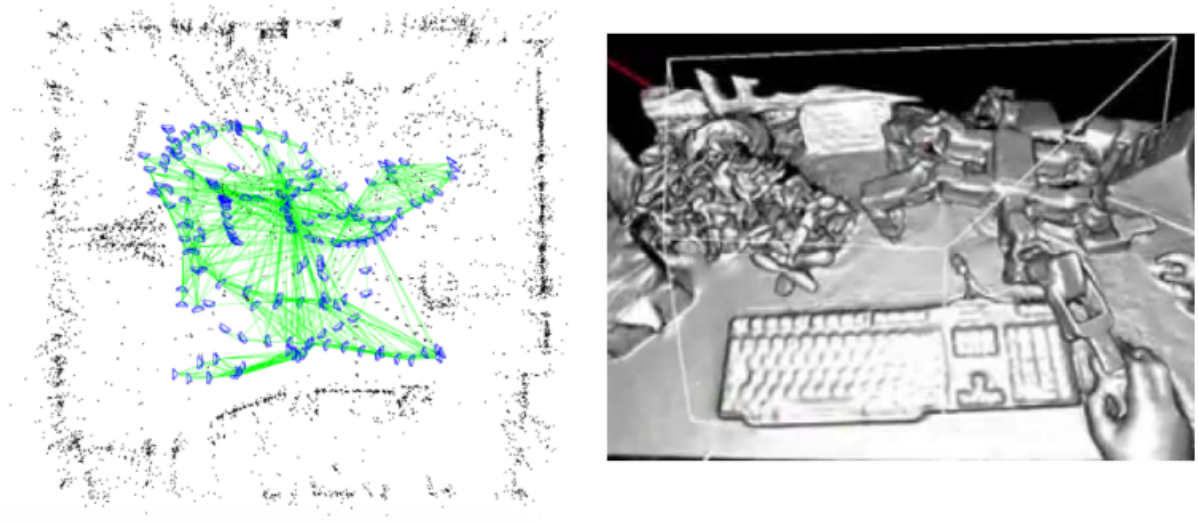


Figure 2: Maps built from one camera: sparse features of a room (left) and a dense desktop (right).

### 6.1.3 SLAM and Structure-from-Motion *(slide 5)*

The unknowns and the error are those of lecture 05: camera poses $\mathbf{T}_i$ (world to camera), 3D points $\mathbf{X}_j$, and the reprojection error of every observation. Both problems stem from the same maximum a posteriori formulation; the *setting* differs.

| **Structure-from-motion (lecture 05)** | **SLAM** |
|---|---|
| Unordered set of images, all available from the start | Frames arrive one at a time; solved incrementally as measurements arrive |
| Solved offline, analyzing all images at once | Pose and map needed at every frame, in real time |
| No notion of returning to a place | Returning to a visited place closes a loop and corrects long-term drift |

**Intuition**

SfM is like developing all holiday photos and then assembling the 3D model at home; SLAM is like navigating a city while drawing the map as you walk. Same maths, very different time budget.

### 6.1.4 Running Example: VBR Campus *(slide 6)*

**VBR** (Vision Benchmark in Rome) was recorded hand-held and by car with a global-shutter stereo camera of wide baseline, a 3D LiDAR, an IMU and an RTK-GPS. Its ground truth refines the RTK-GPS with LiDAR bundle adjustment, accurate to $\pm 3$ cm over 1.5 km; its six datasets cover over 40 km. The running example of the lecture is a sequence of around 11 km.

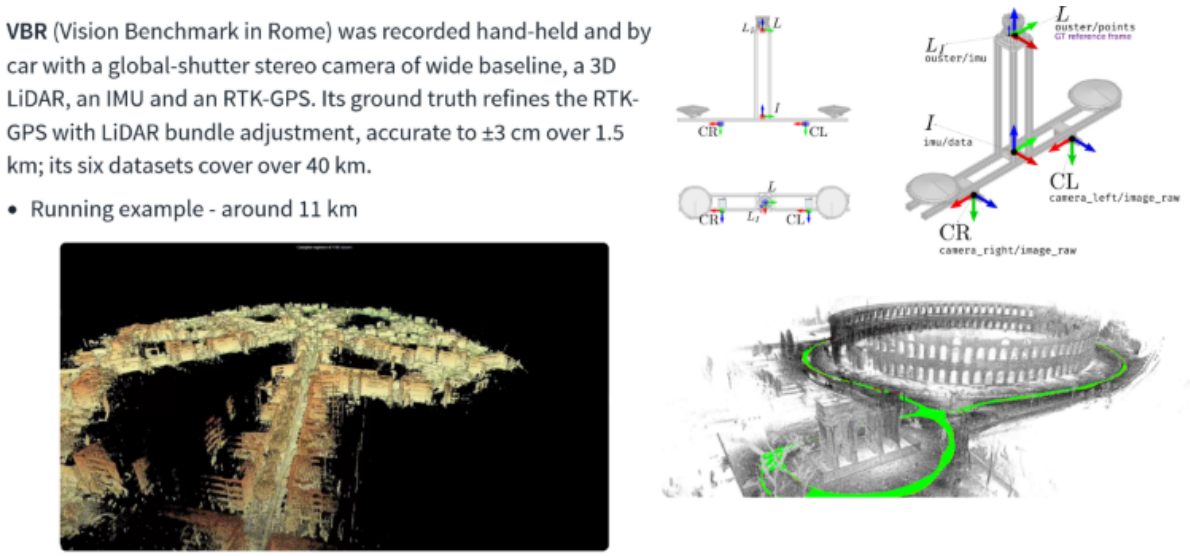



Figure 3: VBR: sensor-rig reference frames (camera left/right CL, CR, LiDAR $L$, IMU $I$) and two point-cloud/map renderings of the Rome sequences.

## 6.2 Filtering: From the Bayes Filter to EKF-SLAM *(slides 7–13)*

### 6.2.1 SLAM as a Graphical Model *(slide 8)*

The camera poses and the map are unknown; the measurements are observed. Without odometry, as with a single camera, a motion model alone links consecutive poses.

**Definition 6.2.1: Full and online SLAM**

*Full SLAM* estimates the whole trajectory and the map, $p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t})$. *Online SLAM* keeps only the current pose,

$$p(\mathbf{x}_t, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) = \int \cdots \int p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t})\, d\mathbf{x}_1 \cdots d\mathbf{x}_{t-1},$$

i.e. it *marginalizes* the past poses, and the integrals are solved recursively one step at a time.

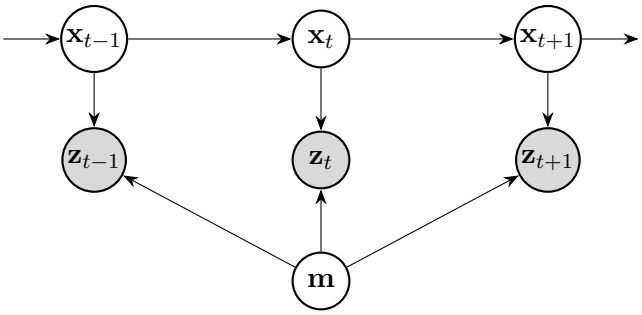



Figure 4: Graphical model of SLAM (slide 8): poses form a Markov chain, the map is static, and each (gray, observed) measurement depends on the current pose and the map. Controls are omitted, as in the monocular case.

**Intuition**

Full SLAM keeps the entire path (a growing problem, solved later by graphs); online SLAM forgets the past path but remembers what it taught us, packed inside a single Gaussian over the current pose and the map. Marginalizing is "summing over everything I no longer care about".

### 6.2.2 The Bayes Filter *(slide 9)*

Let $\mathbf{s}_t = (\mathbf{x}_t, \mathbf{m})$ be the state: camera pose and map.

**Theorem 6.2.2: Recursive Bayes filter**

Let $\mathrm{bel}(\mathbf{s}_t) = p(\mathbf{s}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t})$. Under the Markov assumptions, one frame at a time:

$$\overline{\mathrm{bel}}(\mathbf{s}_t) = \int p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \mathbf{u}_t)\, \mathrm{bel}(\mathbf{s}_{t-1})\, d\mathbf{s}_{t-1} \qquad \text{(prediction)}$$

$$\mathrm{bel}(\mathbf{s}_t) = \eta\, p(\mathbf{z}_t \mid \mathbf{s}_t)\, \overline{\mathrm{bel}}(\mathbf{s}_t) \qquad \text{(update)}$$

where $p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \mathbf{u}_t)$ is the *motion model*, $p(\mathbf{z}_t \mid \mathbf{s}_t)$ the *measurement model* and $\eta$ a normalizer.

*Proof.* By Bayes' rule, $\mathrm{bel}(\mathbf{s}_t) = \eta\, p(\mathbf{z}_t \mid \mathbf{s}_t, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})\, p(\mathbf{s}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})$. Given the state, the current measurement is independent of the past, so the first factor is $p(\mathbf{z}_t \mid \mathbf{s}_t)$. For the second factor, the law of total probability gives $\int p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})\, p(\mathbf{s}_{t-1} \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})\, d\mathbf{s}_{t-1}$; the state is Markov, so $p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \dots) = p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \mathbf{u}_t)$, and $\mathbf{u}_t$ carries no information on $\mathbf{s}_{t-1}$, so the last factor is $\mathrm{bel}(\mathbf{s}_{t-1})$. □

**Intuition**

Two beats, forever: *predict* (move the belief with the motion model, which *widens* it, since motion adds uncertainty) and *update* (multiply by how well each hypothesis explains the new image, which *narrows* it). The one-dimensional example below follows slide 9: a camera on a line.

**Example 6.2.3: Camera on a line**

Prior $\mathcal{N}(0, 1)$. Motion model: move $+1$ with noise variance 1, so the prediction is $\mathcal{N}(1,\, 1 + 1) = \mathcal{N}(1, 2)$. Measurement $z = 2$ with noise variance 1. The posterior is the normalized product of the prediction and the measurement likelihood: $\sigma^2 = (2^{-1} + 1^{-1})^{-1} = 2/3$ and $\mu = \frac{2}{3}\left(\frac{1}{2} \cdot 1 + \frac{1}{1} \cdot 2\right) = 5/3$.

### 6.2.3 Kalman Filter and Extended Kalman Filter *(slide 10)*

With Gaussian noise the belief stays Gaussian, $\mathrm{bel}(\mathbf{s}_t) = \mathcal{N}(\boldsymbol{\mu}_t, \Sigma_t)$. Let the models be $\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{u}_t) + \boldsymbol{\varepsilon}_t$, $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(0, \mathbf{R}_t)$ and $\mathbf{z}_t = h(\mathbf{s}_t) + \boldsymbol{\delta}_t$, $\boldsymbol{\delta}_t \sim \mathcal{N}(0, \mathbf{Q}_t)$. The *extended Kalman filter* (EKF) linearizes both around the current estimate, with Jacobians $\mathbf{G}_t = \partial g / \partial \mathbf{s}\,|_{\boldsymbol{\mu}_{t-1}}$ and $\mathbf{C}_t = \partial h / \partial \mathbf{s}\,|_{\bar{\boldsymbol{\mu}}_t}$.

**Algorithm 6.2.4: Extended Kalman filter step**

**Predict:** $\bar{\boldsymbol{\mu}}_t = g(\boldsymbol{\mu}_{t-1}, \mathbf{u}_t)$, $\quad \bar{\Sigma}_t = \mathbf{G}_t \Sigma_{t-1} \mathbf{G}_t^\top + \mathbf{R}_t$.

**Update:** $\mathbf{S}_t = \mathbf{C}_t \bar{\Sigma}_t \mathbf{C}_t^\top + \mathbf{Q}_t$ (innovation covariance), $\quad \mathbf{K}_t = \bar{\Sigma}_t \mathbf{C}_t^\top \mathbf{S}_t^{-1}$ (Kalman gain),

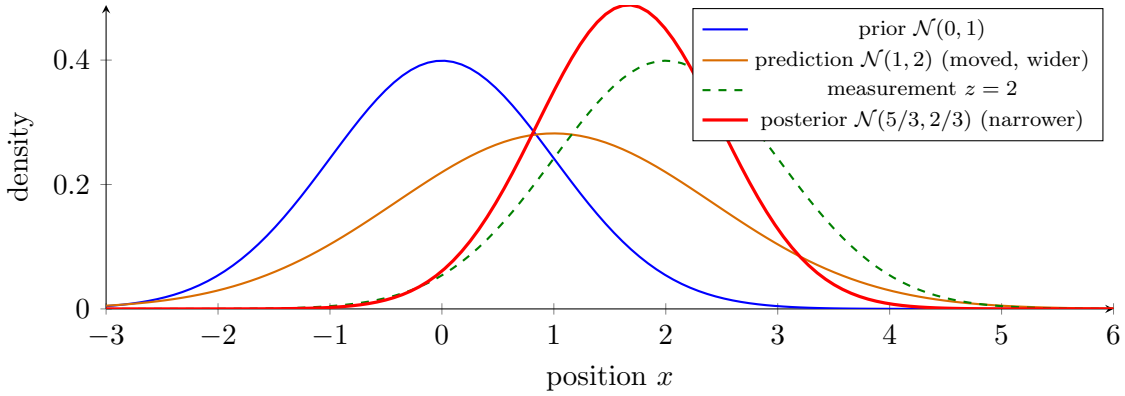



Figure 5: The Bayes filter in one dimension (slide 9): prediction shifts and widens the belief, the measurement narrows it.

$$\boldsymbol{\mu}_t = \bar{\boldsymbol{\mu}}_t + \mathbf{K}_t\big(\mathbf{z}_t - h(\bar{\boldsymbol{\mu}}_t)\big), \quad \Sigma_t = (\mathbf{I} - \mathbf{K}_t\mathbf{C}_t)\bar{\Sigma}_t.$$

**Theorem 6.2.5: Kalman filter (linear case)**

If $g(\mathbf{s},\mathbf{u}) = \mathbf{A}\mathbf{s} + \mathbf{B}\mathbf{u}$ and $h(\mathbf{s}) = \mathbf{C}\mathbf{s}$, the Jacobians are $\mathbf{G}_t = \mathbf{A}$ and $\mathbf{C}_t = \mathbf{C}$, and the EKF equations give the *exact* posterior. This is Kalman's recursive filter for discrete-time linear systems.

*Proof.* The prediction is a linear map of a Gaussian plus Gaussian noise, hence Gaussian with the stated mean and covariance. For the update, $(\mathbf{s}_t, \mathbf{z}_t)$ is jointly Gaussian with $\operatorname{cov}(\mathbf{z}_t) = \mathbf{C}\bar{\Sigma}\mathbf{C}^\top + \mathbf{Q} = \mathbf{S}$ and $\operatorname{cov}(\mathbf{s}_t, \mathbf{z}_t) = \bar{\Sigma}\mathbf{C}^\top$. Conditioning a joint Gaussian gives $\mathbb{E}[\mathbf{s} \mid \mathbf{z}] = \bar{\boldsymbol{\mu}} + \bar{\Sigma}\mathbf{C}^\top\mathbf{S}^{-1}(\mathbf{z} - \mathbf{C}\bar{\boldsymbol{\mu}})$ and $\operatorname{cov} = \bar{\Sigma} - \bar{\Sigma}\mathbf{C}^\top\mathbf{S}^{-1}\mathbf{C}\bar{\Sigma} = (\mathbf{I} - \mathbf{K}\mathbf{C})\bar{\Sigma}$. □

**Intuition**

The gain $\mathbf{K}$ is a trust dial between model and sensor. In 1D with $\bar{\sigma}^2 = 2$ and measurement noise $q = 1$: $K = \bar{\sigma}^2/(\bar{\sigma}^2 + q) = 2/3$, so the posterior mean moves two thirds of the way from the prediction (1) toward the measurement (2), giving $5/3$, exactly Example 6.2.3; the variance shrinks to $(1-K)\bar{\sigma}^2 = 2/3$. For nonlinear $g, h$ the EKF is only an approximation: it converges in the linear case but can *diverge* if nonlinearities are large, because Jacobians are evaluated once, at the current estimate.

### 6.2.4 The EKF-SLAM State *(slide 11)*

EKF-SLAM estimates the camera and the positions of all the points with *one* Gaussian: a single mean and a single joint covariance,

$$\boldsymbol{\mu} = \begin{pmatrix} \mathbf{x}_c \\ \mathbf{y}_1 \\ \vdots \\ \mathbf{y}_N \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} \Sigma_{cc} & \Sigma_{cm} \\ \Sigma_{mc} & \Sigma_{mm} \end{pmatrix}.$$

In MonoSLAM the camera block $\mathbf{x}_c$ has 13 numbers (position 3, orientation quaternion 4, linear velocity 3, angular velocity 3); each point adds 3, so the state has $13 + 3N$ entries and $\Sigma$ has $(13+3N)^2$. The off-diagonal blocks are the *correlations* between the camera and the points and between the points. The filter cycle is: state prediction (the motion model moves the camera

block only), measurement prediction, measurement, data association, update (all blocks change). The camera starts at the origin of its own frame; a new point enters the state when first observed.

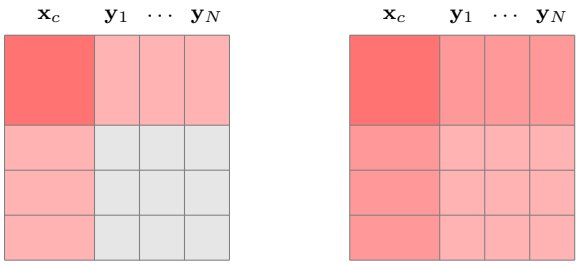



Figure 6: Which blocks of the EKF-SLAM covariance change (red). Left, prediction: only the camera block and its correlations, $\Sigma_{mm}$ is untouched. Right, update: every block changes.

**Proposition 6.2.6: Cost of EKF-SLAM**

The prediction touches only $\Sigma_{cc}$ and $\Sigma_{cm}$ (the Jacobian of $g$ is $\mathrm{diag}(\mathbf{G}_c, \mathbf{I})$), but the update has a dense gain $\mathbf{K}$, so it modifies the *whole* covariance: $O(N^2)$ time per step and $O(N^2)$ memory for $N$ points.

*Proof.* Since $g$ moves only the camera, $\mathbf{G}_t = \mathrm{diag}(\mathbf{G}_c, \mathbf{I})$, giving $\bar{\Sigma}_{cc} = \mathbf{G}_c \Sigma_{cc} \mathbf{G}_c^\top + \mathbf{R}$, $\bar{\Sigma}_{cm} = \mathbf{G}_c \Sigma_{cm}$ and $\bar{\Sigma}_{mm} = \Sigma_{mm}$. In the update, $\Sigma \leftarrow \Sigma - \mathbf{K}\mathbf{S}\mathbf{K}^\top$ with $\mathbf{K} = \bar{\Sigma}\mathbf{C}^\top \mathbf{S}^{-1}$; because all points are correlated with the camera, $\mathbf{K}$ has no zero blocks and the rank-update is dense. □

**Intuition**

Numbers: with $N = 100$ points the state has 313 entries and $\Sigma$ about 98,000 numbers: trivial. With $N = 10{,}000$ points, $30{,}013^2 \approx 9 \times 10^8$ entries, about 7.2 GB in double precision, *and* touched at every frame. Correlations are a feature (observing one point improves the others) but they are what makes the filter expensive.

### 6.2.5 Example: MonoSLAM *(slide 12)*

**MonoSLAM** is EKF-SLAM with a single camera: it recovers the 3D trajectory of a camera moving rapidly through an unknown scene, at 30 Hz on a standard PC, and drives live augmented reality from a hand-held camera.

- **Motion model:** constant linear and angular velocity, with unknown accelerations as Gaussian noise; a hand-held camera has no control input. With $\mathbf{x}_c = (\mathbf{r}, \mathbf{q}, \mathbf{v}, \boldsymbol{\omega})$: $\mathbf{r} \leftarrow \mathbf{r} + (\mathbf{v} + \mathbf{V})\Delta t$, $\mathbf{q} \leftarrow \mathbf{q} \otimes \mathbf{q}\big((\boldsymbol{\omega} + \boldsymbol{\Omega})\Delta t\big)$, $\mathbf{v} \leftarrow \mathbf{v} + \mathbf{V}$, $\boldsymbol{\omega} \leftarrow \boldsymbol{\omega} + \boldsymbol{\Omega}$, with $\mathbf{V}, \boldsymbol{\Omega}$ Gaussian impulses.
- **Measurement model:** pinhole projection of each point, with radial distortion.
- **Active search:** the innovation covariance $\mathbf{S}$ defines an ellipse around each predicted feature, and only these ellipses are searched, for at most 10 or 12 features per frame.
- About 100 features at 30 Hz take 19 ms of the 33 ms of a frame.

**Intuition**

Instead of scanning the whole image for each of 100 features, the filter says "it must be about *here*, give or take this much". Searching a few small ellipses is why a 1990s-style EKF reaches 30 Hz; and the ellipse shrinks as the map is better known.

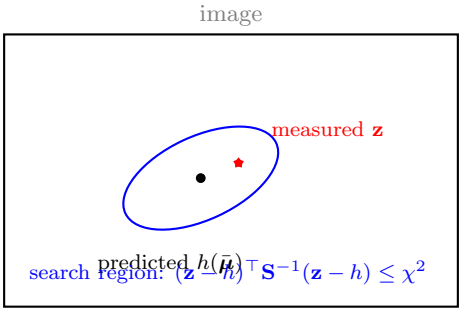



Figure 7: Active search (slide 12): the EKF predicts where a feature should appear and how uncertain that prediction is; only the ellipse is searched.



Search ellipses in a wide-angle image (top); predicted camera paths of the motion model (bottom)

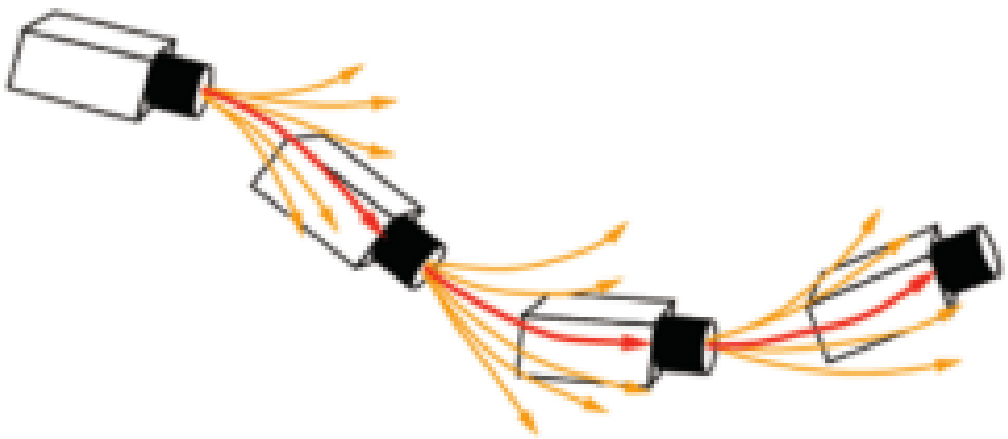


Figure 8: MonoSLAM: search ellipses in a wide-angle image (top) and predicted camera paths of the motion model (bottom).

### 6.2.6 Limits of Filtering *(slide 13)*

- **Cost:** every update touches the whole covariance, $O(N^2)$ per step and in memory. The map must stay small: about 100 features in MonoSLAM at 30 Hz.
- **Linearization:** Jacobians are evaluated once, at the current estimate, and never recomputed; accuracy can fall well below bundle adjustment, and the EKF can diverge if nonlinearities are large.
- **Past poses:** all poses but the current one are marginalized after every frame; what they contributed survives only in the Gaussian over the current pose and the map.

**Proposition 6.2.7: Marginalization creates fill-in**

In information form $\Lambda = \Sigma^{-1}$, partition the variables as $(\mathbf{a}, \mathbf{b})$. Marginalizing out $\mathbf{b}$ gives the information matrix $\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ (Schur complement). Hence all variables that were connected to $\mathbf{b}$ become directly connected to each other.

*Proof.* For a Gaussian with information matrix $\Lambda$, the marginal covariance of $\mathbf{a}$ is the corresponding block of $\Lambda^{-1}$, which by the block-inversion formula equals $(\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba})^{-1}$. The term $\Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ is non-zero in block $(k, l)$ whenever $k$ and $l$ are both neighbours of $\mathbf{b}$. □

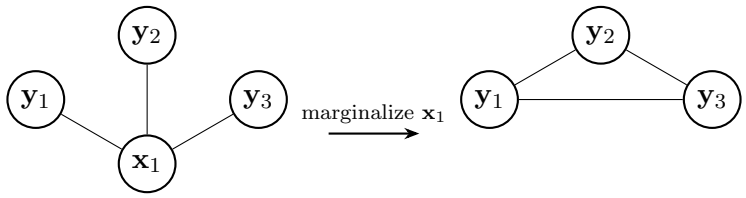



Figure 9: Marginalizing a pose that saw three points turns a sparse star into a dense clique: past poses cannot be dropped for free.

**Intuition**

Filtering throws away the past poses, and the price of forgetting is a dense belief over everything they saw. Graph-based SLAM (next section) keeps the past poses as explicit variables, so the system stays sparse and can be *relinearized*.

## 6.3 Graph-Based SLAM: Factor Graphs and Least Squares *(slides 14–22)*

### 6.3.1 Graph-Based SLAM *(slide 15)*

Graph-based SLAM keeps *every keyframe pose* and represents the problem as a graph. Every node is a camera pose or a map point; every edge is a spatial constraint between two nodes, given by a (uncertain) measurement. Observing a previously seen area creates constraints between non-successive poses. The **front end** builds the graph from raw data; the **back end** finds the node configuration that minimizes the error of all constraints.

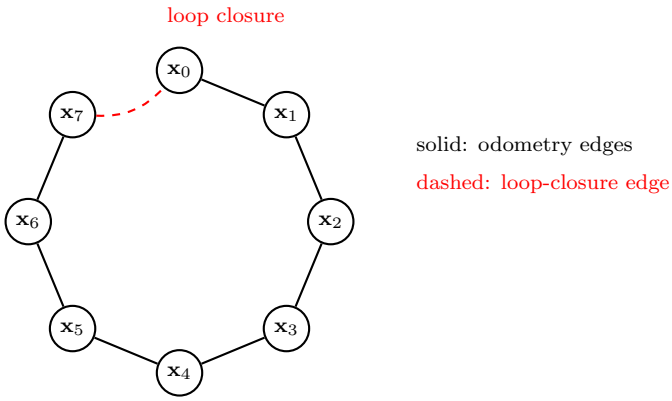



Figure 10: A pose graph (slide 15): odometry links consecutive poses; recognizing a visited place adds a constraint between non-successive poses.

### 6.3.2 Factor Graphs *(slide 16)*

**Definition 6.3.1: Factor graph**

A *factor graph* is a bipartite graph with *variable* nodes (circles: the unknown poses and points) and *factor* nodes (squares). Each factor $\phi_k$ is joined only to the variables $\Theta_k$ it depends on, and the global function is the product

$$\phi(\Theta) = \prod_k \phi_k(\Theta_k).$$

Given the measurements $\mathcal{Z}$, the posterior is proportional to such a product: $p(\Theta \mid \mathcal{Z}) \propto \prod_k \phi_k(\Theta_k)$.

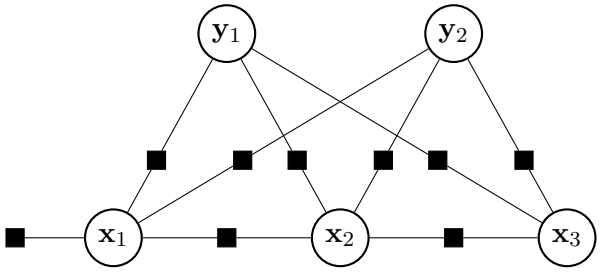



Figure 11: Toy factor graph (redrawn in the spirit of Dellaert and Kaess, Fig. 1.3): three poses, two points, nine factors, namely one prior, two motion factors, and one factor per measurement (six).

**Intuition**

Think of a factor as "one piece of evidence", and its edges as the unknowns it talks about. A GPS reading talks about one pose; an odometry reading about two consecutive poses; a feature observation about one pose and one point. The factor graph is just the table of contents of who-constrains-whom, which will turn out to be exactly the sparsity pattern of the linear system.

### 6.3.3 Visual SLAM as a Factor Graph *(slide 17)*

The variables are the camera poses $\mathbf{T}_i$ and the points $\mathbf{X}_j$. Each observation adds one *reprojection factor*, a Gaussian on the residual of lecture 05:

$$\phi_{ij}(\mathbf{T}_i, \mathbf{X}_j) \propto \exp\Big( -\tfrac{1}{2}\|\mathbf{r}_{ij}\|^2_{\Sigma_{ij}^{-1}} \Big), \qquad \mathbf{r}_{ij} = \pi(\mathbf{T}_i \mathbf{X}_j) - \mathbf{u}_{ij},$$

with $\Sigma_{ij}$ the covariance of the pixel noise. A *prior* factor on $\mathbf{T}_1$ anchors the solution in space; with one camera the scale stays free unless one distance is fixed too (lecture 05). With these Gaussian factors, the most probable variables are the bundle adjustment solution of lecture 05.

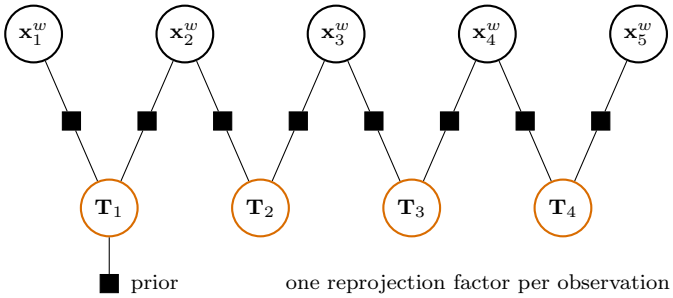



Figure 12: Visual SLAM factor graph (slide 17): each pose sees two neighbouring points.

**Intuition**

Bundle adjustment is not a different algorithm from SLAM's back end: it is the MAP estimate of this graph. The prior on $\mathbf{T}_1$ pins the *gauge*: the whole reconstruction could otherwise slide or rotate in space (and, for one camera, scale) with no change in error.

### 6.3.4 The Pose Graph *(slide 18)*

The pose graph keeps only the poses. Each edge is a measured *relative pose* between two keyframes $i$ and $j$:

- **Odometry edge:** between consecutive keyframes, from the motion estimated between them.
- **Loop-closure edge:** between non-successive keyframes that observe the same part of the scene, a virtual measurement of keyframe $j$ seen from keyframe $i$.

Each edge carries a measured relative pose $\mathbf{Z}_{ij}$ and an information matrix $\Omega_{ij}$. Eliminating the points exactly would connect every pair of poses that saw a common point (Proposition 6.2.7); the pose graph is kept sparse *by construction*, by using only a selected set of relative-pose edges.

### 6.3.5 From Factors to Least Squares *(slide 19)*

Each factor is a Gaussian in its residual, $\phi_k \propto \exp(-\frac{1}{2}\mathbf{e}_k^\top \Omega_k \mathbf{e}_k)$.

**Theorem 6.3.2: MAP on a factor graph is nonlinear least squares**

The most probable variables minimize a sum of squared Mahalanobis errors:

$$\Theta^\star = \arg\max_\Theta \prod_k \phi_k(\Theta_k) = \arg\min_\Theta \; F(\Theta), \qquad F(\Theta) = \sum_k \mathbf{e}_k(\Theta_k)^\top \Omega_k \, \mathbf{e}_k(\Theta_k),$$

where $\mathbf{e}_k$ is the residual of factor $k$ (predicted minus actual measurement) and $\Omega_k$ the inverse covariance of the measurement. The sum runs over prior, motion and measurement terms alike.

*Proof.* $\log \prod_k \phi_k = \sum_k \log \phi_k = \text{const} - \frac{1}{2}\sum_k \mathbf{e}_k^\top \Omega_k \mathbf{e}_k$, and log is monotone, so maximizing the product is minimizing the sum. □

**Intuition**

Logs turn products into sums, Gaussians into parabolas. Worked number: two measurements of the same 1D position, 10.0 with $\sigma = 1$ ($\Omega = 1$) and 10.6 with $\sigma = 0.5$ ($\Omega = 4$). Minimizing $1\,(x-10)^2 + 4\,(x-10.6)^2$ gives $x = \frac{1\cdot 10 + 4\cdot 10.6}{5} = 10.48$: the more certain reading pulls four times harder. This is bundle adjustment of lecture 05, *weighted*.

### 6.3.6 Error and Information Matrix of an Edge *(slide 20)*

An edge measures the pose of keyframe $j$ seen from keyframe $i$, $\mathbf{Z}_{ij}$. Since each pose maps world to camera, the poses predict it as $\mathbf{T}_i \mathbf{T}_j^{-1}$, which maps camera-$j$ coordinates to camera $i$. The residual compares the two:

$$\mathbf{e}_{ij} = \log\!\left(\mathbf{Z}_{ij}^{-1}\,\mathbf{T}_i \mathbf{T}_j^{-1}\right)^\vee \in \mathbb{R}^6, \qquad F = \sum_{(i,j)} \mathbf{e}_{ij}^\top \Omega_{ij}\, \mathbf{e}_{ij}.$$

Here $\mathbf{Z}_{ij}^{-1}$ maps from $i$ to $j$, so $\mathbf{e}_{ij} = \mathbf{0}$ exactly when the poses agree with the measurement; $\Omega_{ij}$ is the inverse covariance of $\mathbf{Z}_{ij}$ and encodes its uncertainty: the larger $\Omega_{ij}$, the more the edge matters in the optimization.

**Intuition**

Think of every edge as a spring whose rest length is the measurement and whose stiffness is $\Omega_{ij}$. Odometry springs are stiff but there are many of them (errors accumulate); a loop-closure spring is softer but connects two far-away parts of the path. Numbers: $\sigma_{\text{odom}} = 0.1\,\text{m}$ gives $\Omega = 100$, while $\sigma_{\text{loop}} = 1\,\text{m}$ gives $\Omega = 1$: a loop edge is $100\times$ weaker per unit squared error, yet it can still bend a long chain because the chain's accumulated error is spread over hundreds of springs.

### 6.3.7 Gauss-Newton on the Pose Graph *(slide 21)*

Around the current poses, every residual is linearized in the increments of lecture 05, $\mathbf{T}_i \leftarrow \exp(\delta\boldsymbol{\xi}_i^\wedge)\mathbf{T}_i$: $\mathbf{e}_{ij}(\mathbf{x} \boxplus \delta\mathbf{x}) \approx \mathbf{e}_{ij} + \mathbf{J}_{ij}\,\delta\mathbf{x}$.

**Algorithm 6.3.3: Gauss–Newton / Levenberg–Marquardt on the pose graph**

Repeat until convergence, starting from a good initial guess (e.g. chained odometry):

1. $\mathbf{H} = \sum_{ij} \mathbf{J}_{ij}^\top \Omega_{ij} \mathbf{J}_{ij}, \quad \mathbf{b} = \sum_{ij} \mathbf{J}_{ij}^\top \Omega_{ij} \mathbf{e}_{ij}$.
2. Solve the normal equations $\mathbf{H}\,\delta\mathbf{x} = -\mathbf{b}$ (Gauss–Newton), or $(\mathbf{H} + \lambda\mathbf{I})\,\delta\mathbf{x} = -\mathbf{b}$ (Levenberg–Marquardt).
3. Update $\mathbf{x} \leftarrow \mathbf{x} \boxplus \delta\mathbf{x}$. In LM, the damping $\lambda$ is doubled when a step increases the error and halved otherwise.

*Derivation of the normal equations.* Substituting the linearization, $F(\mathbf{x}\boxplus\delta\mathbf{x}) \approx \sum(\mathbf{e}+\mathbf{J}\delta\mathbf{x})^\top\Omega(\mathbf{e}+\mathbf{J}\delta\mathbf{x}) = c + 2\,\mathbf{b}^\top\delta\mathbf{x} + \delta\mathbf{x}^\top\mathbf{H}\,\delta\mathbf{x}$. Setting the gradient $2\mathbf{b} + 2\mathbf{H}\delta\mathbf{x}$ to zero gives $\mathbf{H}\delta\mathbf{x} = -\mathbf{b}$. □

**Proposition 6.3.4: Gauge freedom**

The error $F$ is unchanged if all poses move together, $\mathbf{T}_i \mapsto \mathbf{T}_i\mathbf{S}$ for any rigid $\mathbf{S}$. Hence $\mathbf{H}$ is singular (null space of dimension 6), and the graph is anchored by holding $\mathbf{T}_1$ fixed, as the prior on the first pose did.

*Proof.* $(\mathbf{T}_i\mathbf{S})(\mathbf{T}_j\mathbf{S})^{-1} = \mathbf{T}_i\mathbf{T}_j^{-1}$, so every $\mathbf{e}_{ij}$ is invariant. Differentiating along the 6-parameter family shows $\mathbf{J}\mathbf{v} = \mathbf{0}$ for its tangent directions $\mathbf{v}$, hence $\mathbf{H}\mathbf{v} = \sum \mathbf{J}^\top \Omega \mathbf{J}\mathbf{v} = \mathbf{0}$. □

**Intuition**

Pose graphs only know *relative* information, like a map with distances but no coordinates: you can pick the whole thing up and put it down elsewhere. Fixing one pose nails it to the table. LM damping is a safety belt: a large $\lambda$ takes small, gradient-like steps; a small $\lambda$ takes full Gauss–Newton steps. This handles highly non-convex problems better, and loop closures are exactly where the problem is non-convex.

### 6.3.8 Sparsity of the Normal Equations *(slide 22)*

A residual depends on $\mathbf{T}_i$ and $\mathbf{T}_j$ only, so its Jacobian has two non-zero blocks, $\mathbf{J}_{ij} = [\cdots \ \mathbf{A}_{ij} \ \cdots \ \mathbf{B}_{ij} \ \cdots]$, and each edge adds to *four blocks of* $\mathbf{H}$ *and two of* $\mathbf{b}$:

$$\mathbf{H}_{ii} \mathrel{+}= \mathbf{A}^\top\Omega\mathbf{A}, \ \mathbf{H}_{ij} \mathrel{+}= \mathbf{A}^\top\Omega\mathbf{B}, \ \mathbf{H}_{ji} \mathrel{+}= \mathbf{B}^\top\Omega\mathbf{A}, \ \mathbf{H}_{jj} \mathrel{+}= \mathbf{B}^\top\Omega\mathbf{B}, \qquad \mathbf{b}_i \mathrel{+}= \mathbf{A}^\top\Omega\mathbf{e}, \ \mathbf{b}_j \mathrel{+}= \mathbf{B}^\top\Omega\mathbf{e}.$$

**Proposition 6.3.5: Sparsity pattern of H**

The block $\mathbf{H}_{ij}$ is non-zero if and only if there is an edge between $i$ and $j$. The block pattern of $\mathbf{H}$ is therefore the adjacency matrix of the graph plus the diagonal, and $\mathbf{H}$ is assembled by summing the contribution of each edge.

*Proof.* Off-diagonal blocks only receive the terms $\mathbf{A}^\top\Omega\mathbf{B}$ above, one per edge joining $i$ and $j$; all other products $\mathbf{J}_{kl}^\top\Omega\mathbf{J}_{kl}$ are zero in block $(i, j)$. □

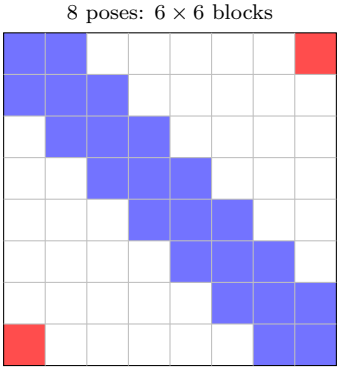



Figure 13: Block sparsity of $\mathbf{H}$ for the ring pose graph of slide 15: diagonal and first off-diagonals come from odometry (blue), the two red blocks come from the single loop closure.

**Intuition**

Worked numbers: 1000 poses with 6 degrees of freedom give $\mathbf{H} \in \mathbb{R}^{6000\times6000}$, i.e. 36 million entries if dense. But there are only about 1000 diagonal and $2 \times 999$ off-diagonal blocks of 36 numbers, roughly 108,000 non-zeros (0.3%). Sparse solvers (sparse Cholesky, conjugate gradients) exploit this, which is why graph-based SLAM scales to the 11 km running example while a dense EKF covariance cannot.

## 6.4 The Visual SLAM Pipeline: ORB-SLAM *(slides 23–32)*

### 6.4.1 Pipeline Overview *(slide 24)*

**ORB-SLAM**, a keyframe-based monocular SLAM system, runs three threads in parallel: **tracking** localizes the camera at every frame and decides when to insert a keyframe; **local mapping** processes each new keyframe and runs bundle adjustment around it; **loop closing** searches for a loop at every keyframe and corrects the drift by pose graph optimization. The *front end* abstracts the images into features and data associations; the *back end* estimates the variables from them (Section 6.3).

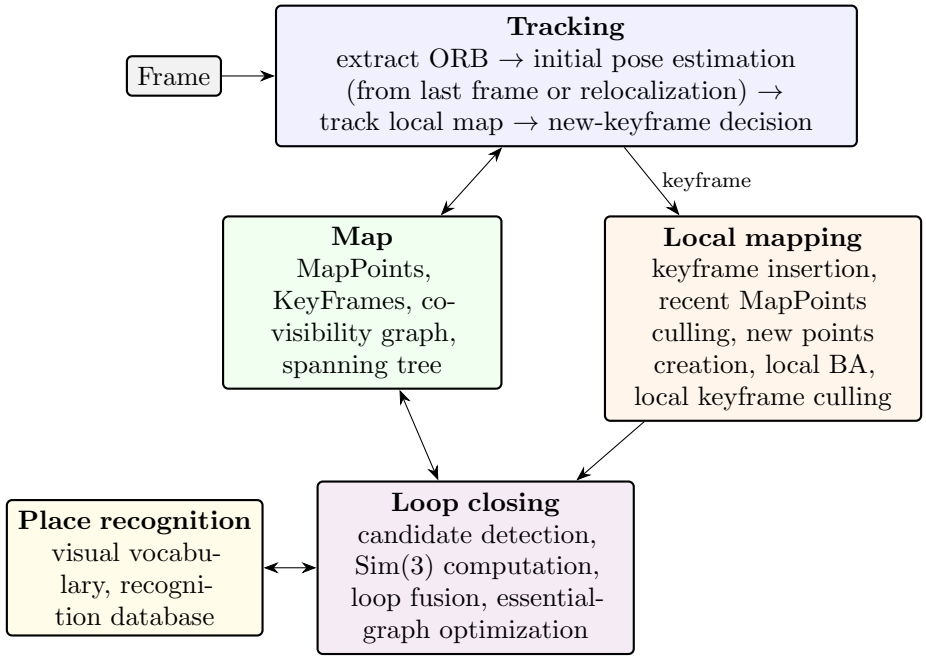



Figure 14: ORB-SLAM threads and data structures (redrawn from the slide 24 block diagram).

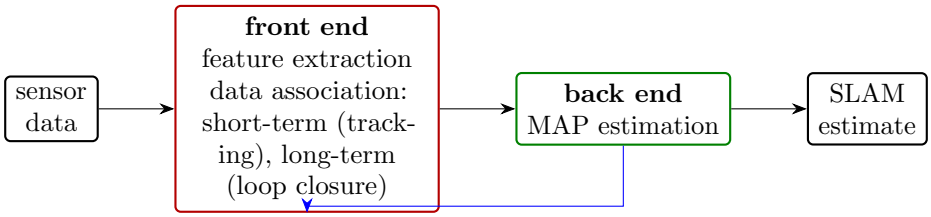



Figure 15: Front end / back end split (slide 24, right): the back end also feeds predictions back to guide data association.

**Intuition**

The split mirrors the lecture: Section 6.3 was the back end (sparse least squares, assumed data associations), and this section is a front end that produces those associations fast enough for 30 frames per second. Three threads exist because the three jobs have different deadlines: tracking must finish within one frame, local mapping within a keyframe interval, loop closing can take longer.

### 6.4.2 ORB Features and Matching *(slide 25)*

ORB-SLAM uses the same features for tracking, mapping, relocalization and loop detection. As in lecture 05, a feature is a keypoint with a descriptor.

- Extraction must take much less than the 33 ms between frames, which excludes SIFT at about 300 ms.
- **ORB**: corners from the FAST detector, given an orientation, found at 8 scale levels a factor 1.2 apart, each with a binary descriptor; the orientation makes it rotation invariant.
- 1000 keypoints per image up to $752 \times 480$ pixels, 2000 on larger images ($1241 \times 376$), spread over a grid.

**Definition 6.4.1: Hamming distance**

For binary descriptors $\mathbf{d}, \mathbf{d}' \in \{0,1\}^n$ the *Hamming distance* is the number of differing bits, $\|\mathbf{d} \oplus \mathbf{d}'\|_1 = \text{popcount}(\mathbf{d} \text{ XOR } \mathbf{d}')$.

**Intuition**

Matching two descriptors is one XOR and one popcount instruction: orders of magnitude cheaper than the Euclidean distance between 128 floats of SIFT. Example with 8 bits: `10110010` vs. `10011010` differ in 2 positions, so $d = 2$. The 8-level pyramid with factor 1.2 covers a total scale range of $1.2^7 \approx 3.6$, which is why ORB matches survive strong zoom changes.

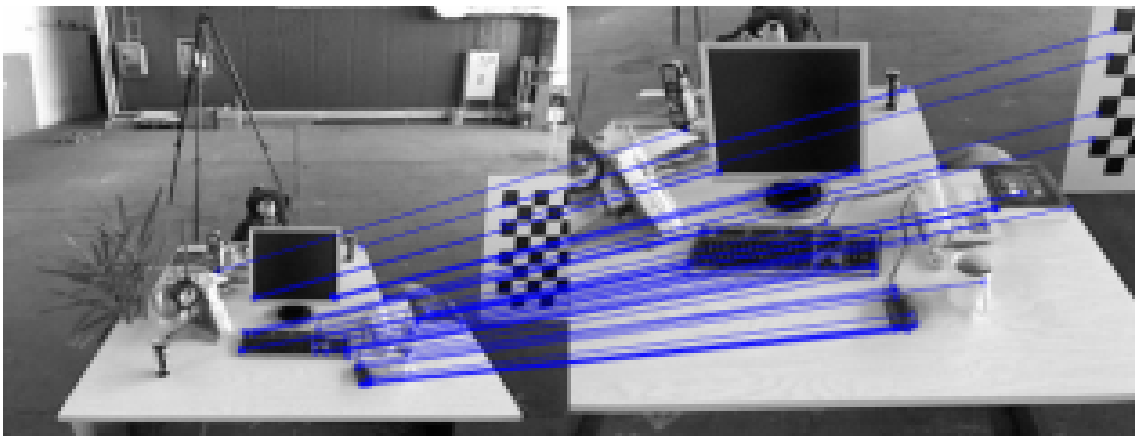



Figure 16: ORB matches between a frame and a keyframe under a strong change of scale.

### 6.4.3 Map Initialization *(slide 26)*

The first map needs the relative pose of two frames and triangulated points, for planar and general scenes alike. Between a reference frame 1 and the current frame 2, two models are fitted in parallel with RANSAC: a homography $\mathbf{x}_2 \simeq \mathbf{H}_{21}\mathbf{x}_1$ (4 points, planar scene) and a fundamental matrix $\mathbf{x}_2^\top \mathbf{F}_{21}\mathbf{x}_1 = 0$ (8 points, general scene), each scored by its symmetric transfer error

$$S_M = \sum_i \Big[\rho_M\big(d^2(\mathbf{x}_1^i, M^{-1}\mathbf{x}_2^i)\big) + \rho_M\big(d^2(\mathbf{x}_2^i, M\mathbf{x}_1^i)\big)\Big].$$

**Algorithm 6.4.2: ORB-SLAM map initialization**

1. Fit $\mathbf{H}_{21}$ and $\mathbf{F}_{21}$ with RANSAC and compute the scores $S_H$, $S_F$.
2. Select the homography if $S_H/(S_H + S_F) > 0.45$ (planar scenes and low parallax); otherwise the fundamental matrix.
3. Recover the motion hypotheses: 8 from the essential matrix or 4 from the homography (lecture 05); triangulate with all of them.
4. If there is no clear winner, *create no map* and try with new frames.
5. Refine the first map by full bundle adjustment; its scale is arbitrary (lecture 05).

**Intuition**

A homography explains almost anything seen with *no parallax* (a plane, or pure rotation), but then depth is ill-defined and triangulation is unreliable. ORB-SLAM prefers to *wait for parallax* rather than start from a wrong planar map; the slide shows two other systems that started wrongly and drifted, while ORB-SLAM, which waited, did not.

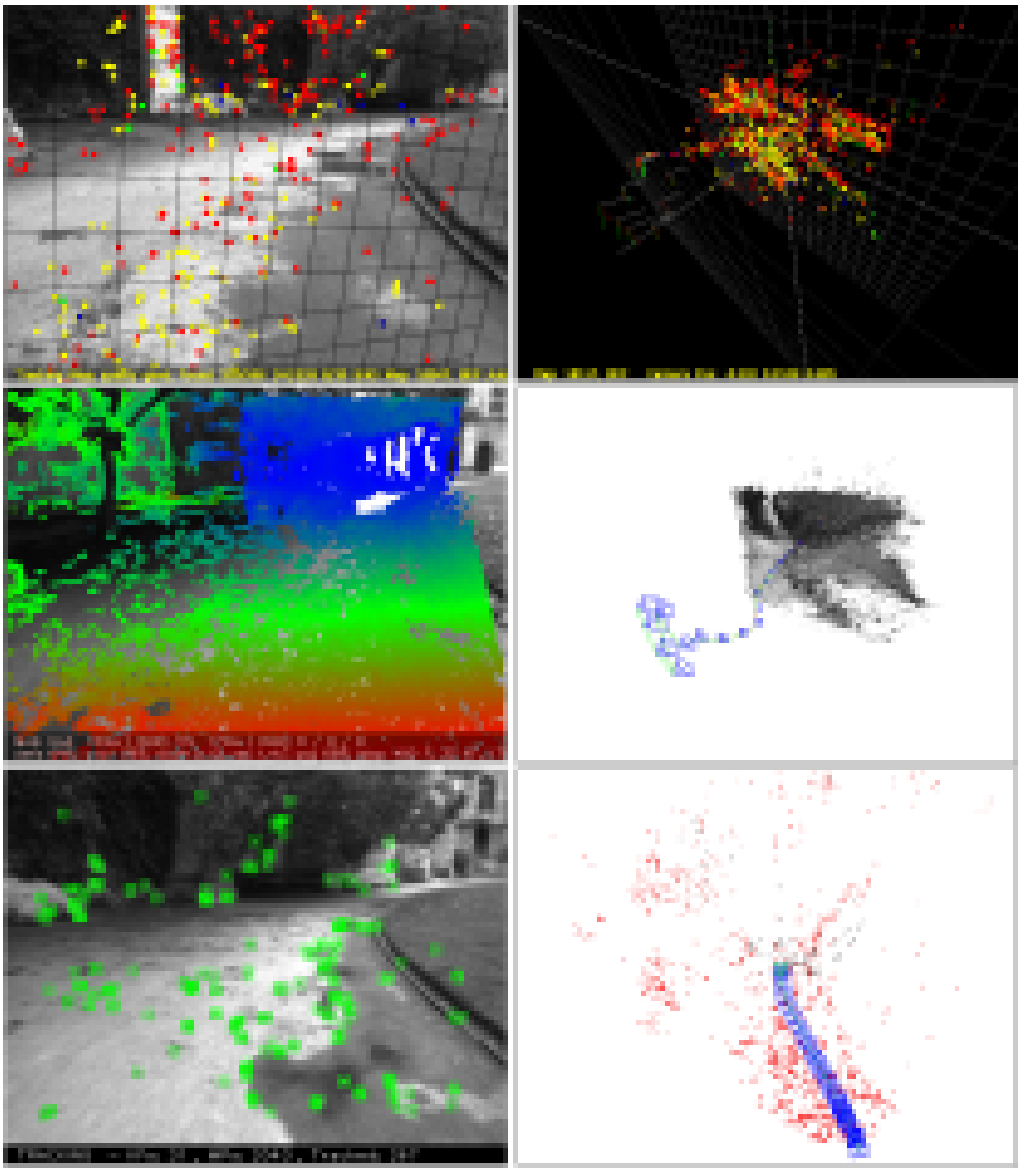


Figure 17: Some time after initialization: two other systems (top, middle) started from a wrong planar map; ORB-SLAM (bottom) waited for parallax.

### 6.4.4 Tracking *(slide 27)*

The tracking thread estimates the pose of every frame:

1. **Prediction:** a constant-velocity motion model, $\mathbf{T}_t^{\text{pred}} = (\mathbf{T}_{t-1}\mathbf{T}_{t-2}^{-1})\,\mathbf{T}_{t-1}$, predicts the pose and guides the search for the points seen in the last frame; if too few are found, a wider search follows.
2. **Local map:** the keyframes that share points with the frame, and their covisible neighbours; their points are projected into the frame and matched near the projection, at the predicted scale.
3. **Motion-only bundle adjustment:** the reprojection error of lecture 05 over the pose alone, points fixed, with a robust (Huber) cost,

$$\mathbf{T}^\star = \arg\min_{\mathbf{T}} \sum_j \rho_{\text{Huber}}\Big(\|\mathbf{u}_j - \pi(\mathbf{T}\mathbf{X}_j)\|^2_{\Sigma^{-1}}\Big).$$

**Intuition**

Only 6 unknowns are optimized, so this runs in a few milliseconds per frame. The Huber cost behaves quadratically for small errors and linearly for large ones, so one wrong match (an outlier with a 50-pixel error) cannot dominate 100 good ones. **PTAM**, a monocular system for small augmented-reality workspaces, predicts with a decaying velocity and tracks coarse to fine: 50 points at the coarsest image scale, then up to 1000.

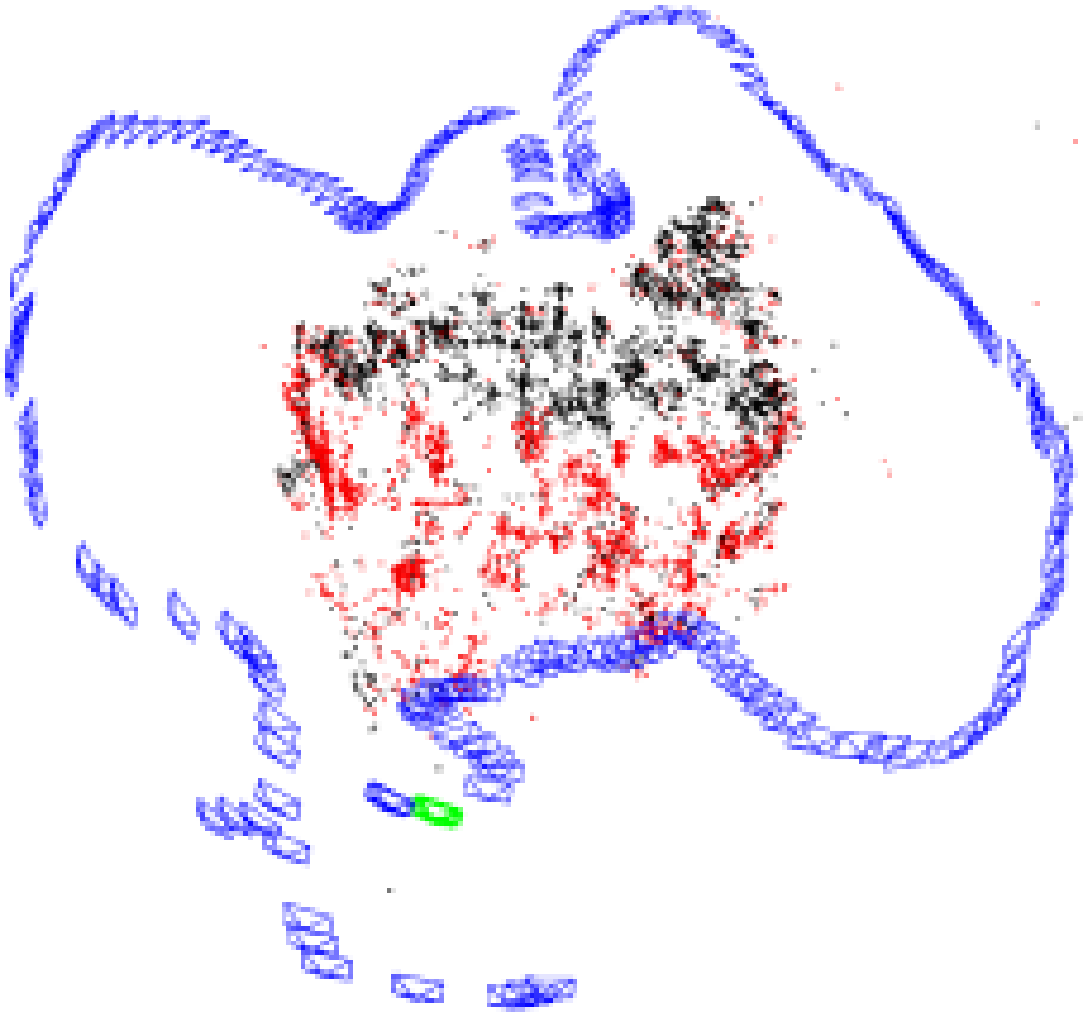


Figure 18: Keyframes (blue), current camera (green), map points (black) and local map (red).

### 6.4.5 Keyframes *(slide 28)*

PTAM splits SLAM into two threads that run in parallel: **tracking** processes every frame in real time (freed from updating the map, it can process each image more thoroughly), and **mapping** works on keyframes only: many video frames are redundant, and a smaller set of useful keyframes is refined by bundle adjustment without strict real-time limits.

- PTAM: a frame becomes a keyframe if tracking is good, more than twenty frames have passed, and the camera is a minimum distance (set by the mean scene depth) from the existing keyframes.
- ORB-SLAM instead inserts keyframes *as fast as possible*, which keeps tracking robust to rotations, and culls the redundant ones in local mapping.

**Intuition**

Keyframes are the "postcards" of the trip: you do not keep every frame of a 30 Hz video (11 km would be hundreds of thousands), only enough views to triangulate the scene. ORB-SLAM's strategy is generous insertion followed by pruning: it is easier to remove a redundant postcard than to recover a view you never kept.

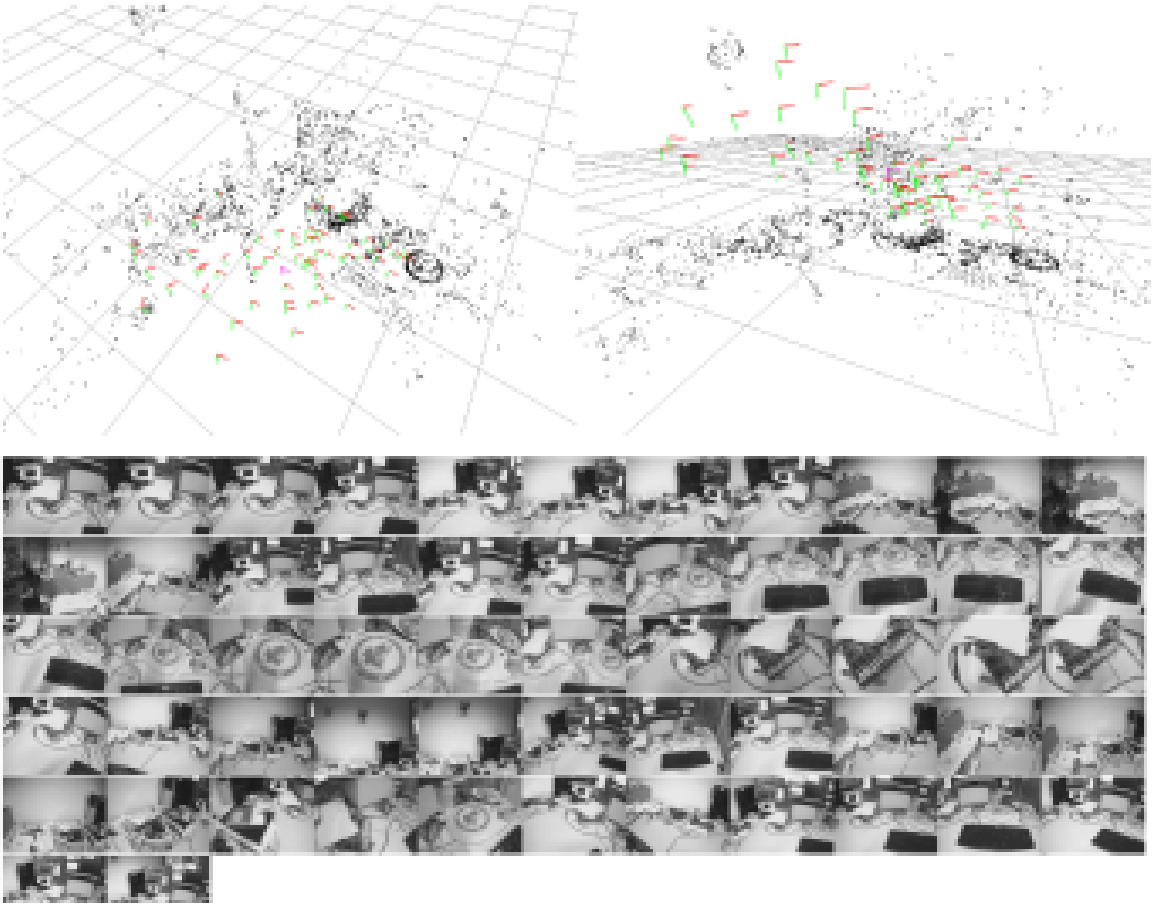


Figure 19: A desk mapped by PTAM: two views of the map with its keyframes (top), and the 57 keyframes (bottom).

### 6.4.6 Local Mapping *(slide 29)*

**Definition 6.4.3: Covisibility graph**

Nodes are keyframes; an edge joins two keyframes that share at least 15 map points, weighted by the number of shared points.

For every new keyframe the local mapping thread:

1. inserts it in the covisibility graph;
2. culls recent points: a point stays only if it is found in more than 25% of the frames predicted to see it, and from at least three keyframes;
3. triangulates new points from unmatched features in covisible keyframes, checking depth, parallax, reprojection error and scale;
4. runs **local bundle adjustment** on the keyframe, its covisible keyframes and all their points; the other keyframes that see these points stay fixed;

5. culls redundant keyframes, whose points are 90% seen by at least three other keyframes.

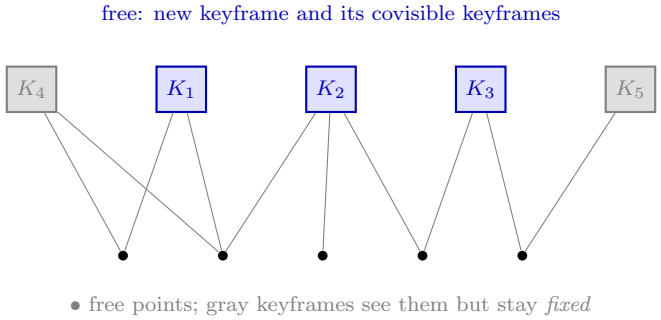



Figure 20: Local bundle adjustment (slide 29, step 4): only a small neighbourhood is optimized; fixed keyframes anchor it to the rest of the map.

**Intuition**

The covisibility graph answers "who looks at the same things as me?". Optimizing only that neighbourhood keeps the cost bounded while the map grows to kilometres: *local* BA is constant-time per keyframe, unlike EKF-SLAM whose cost grows with the whole map. Fixing the outer keyframes plays the role of the prior on $\mathbf{T}_1$ for the local problem.

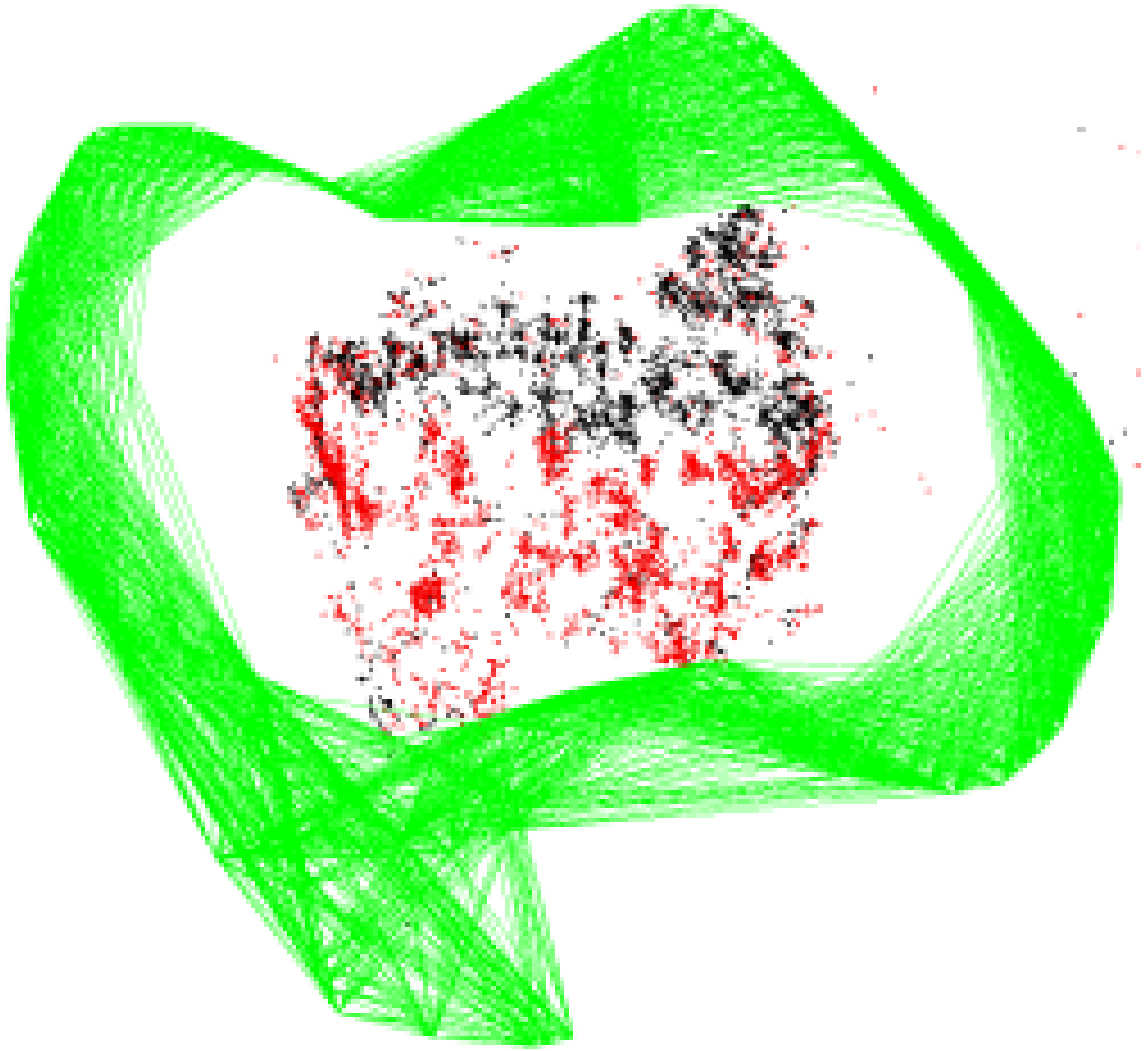


Figure 21: Covisibility graph of an indoor handheld sequence (TUM RGB-D).

### 6.4.7 Loop Detection with Bags of Binary Words *(slide 30)*

A loop is detected by *appearance*: each image is compared with all the past ones, fast enough for every frame.

- **Vocabulary:** binary descriptors clustered offline into a tree of 10 branches and 6 levels whose leaves are the visual words; an image becomes a vector of word weights.
- **Inverse index:** for each word, the images that contain it, so a query visits only images sharing words with it.
- **Checks:** the score is normalized by that of the previous image; a match must agree with previous matches and be supported by a fundamental matrix with at least 12 RANSAC inliers.
- ORB-SLAM uses ORB words, groups candidates by covisibility rather than by time, and accepts a loop after three consecutive consistent candidates. 21.6 ms per image on average, over 26292 images.

**Proposition 6.4.4: Why a hierarchical vocabulary is fast**

A tree with branching factor $k$ and $L$ levels has $k^L$ leaves (words), but quantizing a descriptor needs only $kL$ comparisons.

*Proof.* At each level the descriptor is compared with the $k$ children of the current node and descends into the closest one; there are $L$ levels. □

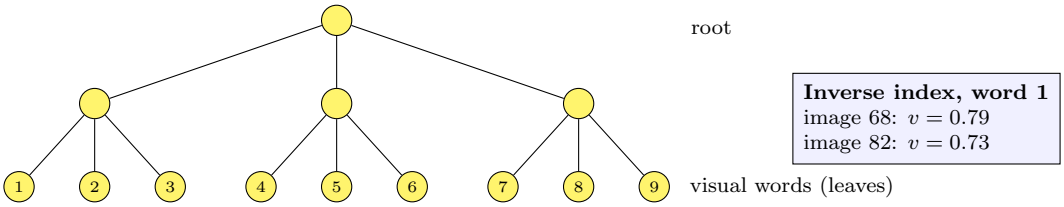



Figure 22: Vocabulary tree and inverse index (slide 30; the sketch shows $k = 3,\ L = 2$, the system uses $k = 10,\ L = 6$).

**Intuition**

Numbers: 10 branches and 6 levels give $10^6 = 1{,}000{,}000$ words, but a descriptor is quantized with only $10 \times 6 = 60$ comparisons. The inverse index is the book index idea: to find pages about "Cholesky" you do not read the whole book. Normalizing by the score of the previous image avoids false alarms in places that simply look similar to everything (corridors). The score between two images lies between 0 and 1.

### 6.4.8 Loop Correction and Scale Drift *(slide 31)*

One camera cannot observe scale, so the scale drifts too: a monocular map drifts in **seven** degrees of freedom (rotation, translation and scale). A pose graph over SE(3) closes the loop but *keeps the scale drift*. The poses become similarities (lecture 01).

**Definition 6.4.5: Similarity transformation** Sim(3)

$\mathbf{S} = \begin{pmatrix} s\mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix}$, with $\mathbf{R} \in SO(3)$, $\mathbf{t} \in \mathbb{R}^3$, $s > 0$; it acts on points as $\mathbf{X} \mapsto s\mathbf{R}\mathbf{X} + \mathbf{t}$ and has 7 degrees of freedom. The pose-graph residual has the same form as before, $\mathbf{e}_{ij} = \log(\mathbf{Z}_{ij}^{-1}\mathbf{S}_i\mathbf{S}_j^{-1})^\vee \in \mathbb{R}^7$.

- Odometry edges keep their measured motion with scale 1; the loop edge carries the measured change of scale.
- **ORB-SLAM:** the loop similarity comes from 3D–3D point matches, by RANSAC and Horn's method; duplicated points are fused; the pose graph over Sim(3) is optimized on the *essential graph*: a spanning tree, the covisibility edges with at least 100 shared points, and the loop edges.

**Intuition**

Numbers: if after a loop the end of the trajectory is estimated at relative scale 1.10 with respect to the start, the loop edge says so, and the Sim(3) optimization spreads $\ln 1.10 \approx 0.0953$

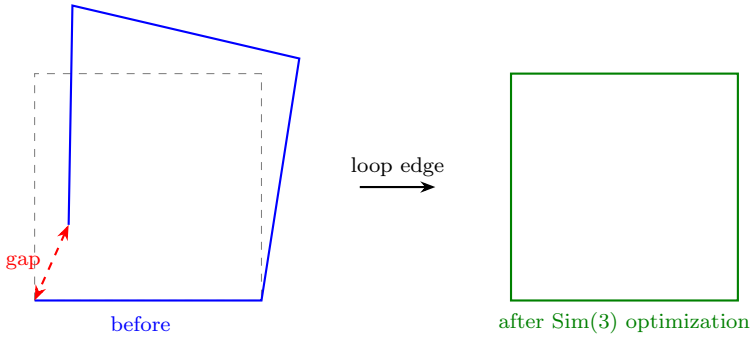



Figure 23: Loop closure (sketch of the slide 31 experiment): drift in rotation, translation and scale leaves the loop open; the loop edge and its scale are distributed along the cycle.

over, say, 100 keyframes, about 0.00095 of log-scale each. A SE(3) optimization has no scale variable, so it can bend the path but the scale error stays: the slide shows the 6 DoF result still distorted while the 7 DoF result matches the aerial photo.

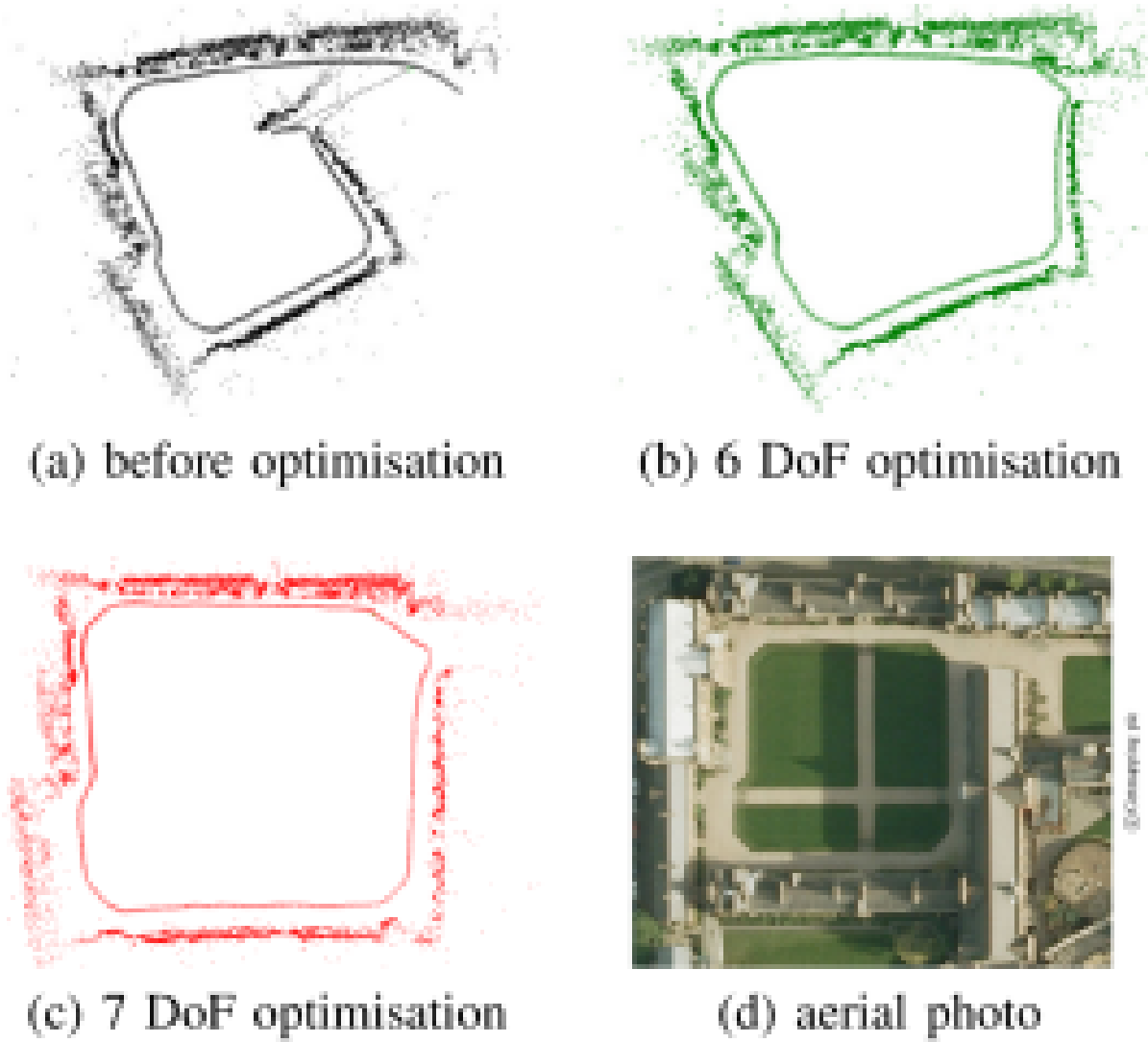



Figure 24: A square circled by a hand-held camera: (a) before optimization, (b) over SE(3) (6 DoF), (c) over Sim(3) (7 DoF), (d) aerial photo.

### 6.4.9 Relocalization *(slide 32)*

When tracking is lost, after an occlusion or an abrupt motion, the camera is localized again in the existing map:

1. the frame is converted into a bag of words, and the place recognition database returns candidate keyframes;
2. for each candidate, ORB features are matched to its map points, and RANSAC with PnP (lecture 05) estimates a camera pose;
3. a pose with enough inliers is refined and guides the search for more matches; with enough inliers, tracking resumes.

On indoor handheld sequences with ground truth (TUM RGB-D), 78.4% of 2769 frames are relocalized in a map built from the first 30 s; with people walking through the scene, 77.9% of 859 frames.

**Intuition**

Relocalization reuses everything already built: the same bag-of-words database that detects loops here answers "where am I?" from a single image. It is a *global* localization problem (no prior pose), solved by PnP inside RANSAC, which tolerates the moving people that spoil many of the matches.

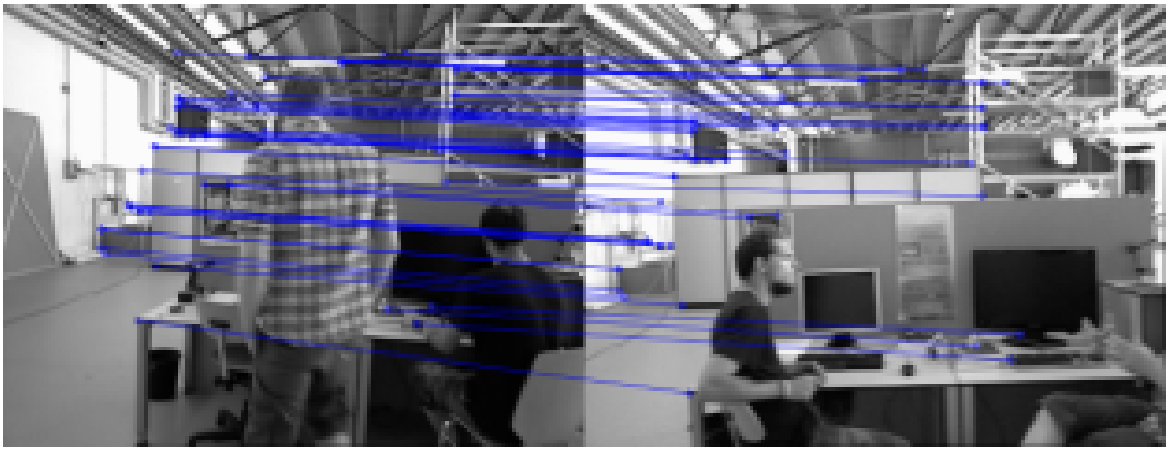


Figure 25: A relocalization among moving people: ORB matches between the frame and a keyframe.

## 6.5 Research Directions *(slides 33–40)*

### 6.5.1 Continuous Time: Rolling-Shutter Cameras *(slide 34)*

A rolling-shutter camera exposes every line of the image for a different period, each one more delayed than the last. When the camera moves during the exposure, the image appears distorted and skewed. So far every image had one pose, as with a global shutter, where all pixels are exposed over the same duration. Earlier work removed the distortion first, to recreate a global-shutter image. On a simulated monocular sequence, the loop-closure position error is 0.0472 of the distance travelled when time is modelled as for a global shutter, and 0.0057 when the rolling shutter is modelled.

**Intuition**

The numbers: 4.7% versus 0.57% of the path length, an 8× reduction ($0.0472/0.0057 \approx 8.3$) just from admitting that the rows of an image are taken at different times. Each scanline is a tiny camera with its own timestamp, so the *trajectory*, not a pose per image, becomes the unknown.

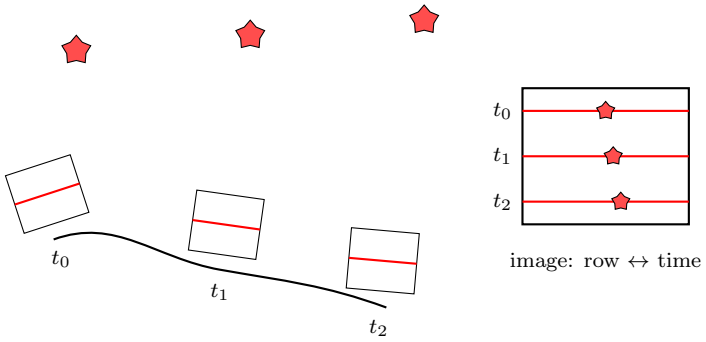



Figure 26: Each scanline is a camera with its own time along the trajectory (redrawn from slide 34); together they form the image.

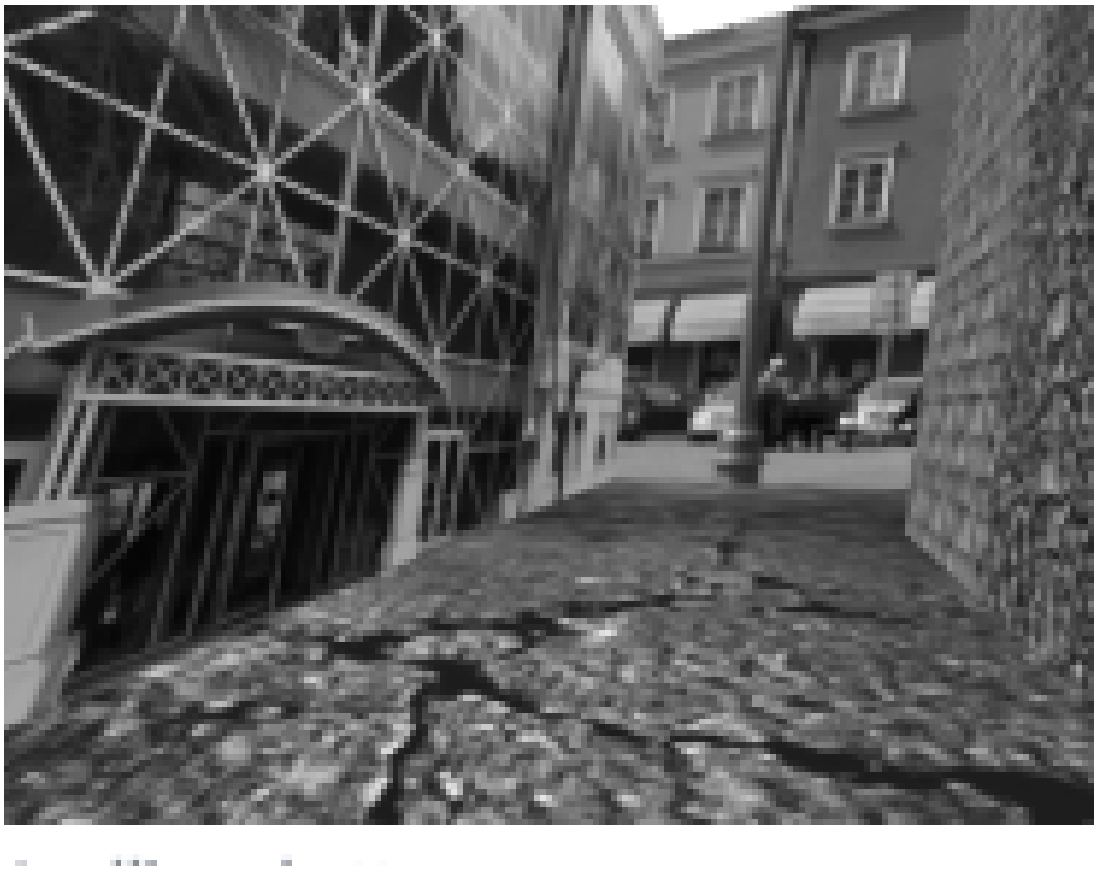


Figure 27: A rolling-shutter warp (distorted and skewed building facade).

### 6.5.2 Continuous Time: Cumulative B-Splines on SE(3) *(slide 35)*

The trajectory becomes a continuous function of time, a *cumulative cubic B-spline* built with the Lie algebra (lecture 05). Control poses $\mathbf{T}_i$ sit at uniform times $t_i$, $\Delta t$ apart. For $t \in [t_i, t_{i+1})$ let $u = (t - t_i)/\Delta t \in [0, 1)$; the four poses $\mathbf{T}_{i-1}, \dots, \mathbf{T}_{i+2}$ give

$$\mathbf{T}(t) = \exp\big(\tilde{B}_3(u)\,\boldsymbol{\delta}_3\big)\exp\big(\tilde{B}_2(u)\,\boldsymbol{\delta}_2\big)\exp\big(\tilde{B}_1(u)\,\boldsymbol{\delta}_1\big)\,\mathbf{T}_{i-1}, \qquad \boldsymbol{\delta}_j = \log\big(\mathbf{T}_{i-1+j}\mathbf{T}_{i-2+j}^{-1}\big),$$

with cumulative basis functions

$$\tilde{B}_1 = \tfrac{1}{6}(5 + 3u - 3u^2 + u^3), \quad \tilde{B}_2 = \tfrac{1}{6}(1 + 3u + 3u^2 - 2u^3), \quad \tilde{B}_3 = \tfrac{1}{6}u^3.$$

(Spline written for world-to-camera poses with left-multiplied increments.) $\boldsymbol{\delta}_j$ is the increment from control pose $j-1$ to $j$, and the trajectory is twice continuously differentiable. Each pixel is projected with the pose at its own time: a point on image row $v$ is observed at $t = t_0 + \frac{v}{h}\, t_{\mathrm{ro}}$, $h$ being the image height and $t_{\mathrm{ro}}$ the frame readout time. The control poses are the unknowns of the least squares, optimized over a sliding window.

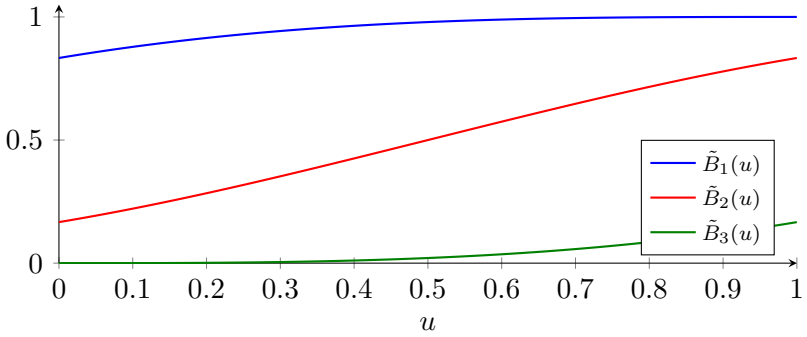



Figure 28: Cumulative cubic B-spline basis functions: at $u = 0$ the weights are $(5/6, 1/6, 0)$ and at $u = 1$ they are $(1, 5/6, 1/6)$, which equals the $u = 0$ weights of the next interval after shifting the control poses by one, so the curve is continuous.

**Intuition**

A spline is a flexible ruler pinned at control poses: you do not store a pose per row, only a handful of control poses per window, and any timestamp inside gives a smooth pose. Advantage: IMU, rolling-shutter and event-camera measurements at arbitrary timestamps can all be written as functions of the same few unknowns. Further reading: a survey of continuous-time state estimation in robotics (splines and Gaussian processes), Talbot et al., T-RO 2025.

### 6.5.3 Active SLAM *(slide 36)*

SLAM is *passive*: it does not guide the motion. **Active SLAM** plans and controls the motion of the robot to build the most accurate and complete model of the environment. It is a decision between two opposite goals, exploring new areas or revisiting those already seen (the *exploration–exploitation dilemma*). With high uncertainty, places where a loop could close carry more information than new frontiers.

**Algorithm 6.5.1: Active SLAM loop**

Repeat until a stopping condition is met: (1) identify candidate actions; (2) compute the utility of each; (3) select and execute the best.

The utility is the expected uncertainty of the pose and the map. For a Gaussian belief whose covariance $\Sigma \in \mathbb{R}^{n\times n}$ has eigenvalues $\lambda_1, \ldots, \lambda_n$, the *D-optimality* criterion measures the volume of the uncertainty ellipsoid,

$$\text{D-opt}(\Sigma) = \exp\Big(\tfrac{1}{n}\sum_{k=1}^{n} \log \lambda_k\Big) = \det(\Sigma)^{1/n},$$

to be minimized.

**Intuition**

Worked number: eigenvalues $(4, 1)$ give $\det^{1/2} = 2$; after an informative view that shrinks the long axis to $(1, 1)$ the criterion is 1. The geometric mean of the eigenvalues is used because it rewards shrinking *any* axis, not only the largest one.

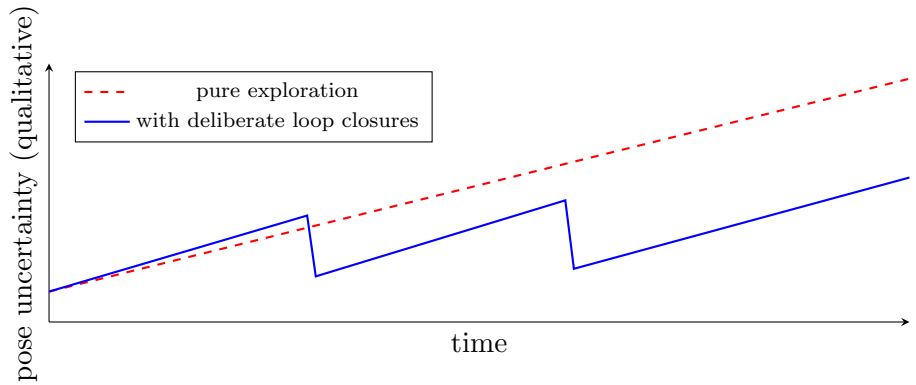



Figure 29: Exploration–exploitation (qualitative sketch): exploring grows uncertainty; revisiting a known place closes a loop and resets it.

### 6.5.4 Active SLAM with an Active Stereo Head *(slide 37)*

The work that named active SLAM: a wheeled robot carries a stereo head with four degrees of freedom. An EKF estimates the robot and a map of point features, and the head fixates one feature at a time; it decides where to look, not where to move. For each visible feature $i$ the EKF predicts the measurement and its innovation covariance $\mathbf{S}_i$. The head fixates the feature whose position is hardest to predict, the one with the largest $3\sigma$ ellipsoid:

$$i^\star = \arg\max_i \ \mathrm{Vol}_{3\sigma}(\mathbf{S}_i) = \arg\max_i \ \det(\mathbf{S}_i)^{1/2}.$$

**Intuition**

It is MonoSLAM's search ellipse turned into a decision: look at the thing you are *least sure about*, because a measurement there reduces uncertainty the most. The ellipsoid volume is $\frac{4}{3}\pi\, 3^3 \sqrt{\det \mathbf{S}_i}$ for a 3D innovation, so ranking by $\det \mathbf{S}_i$ is the same as ranking by volume.

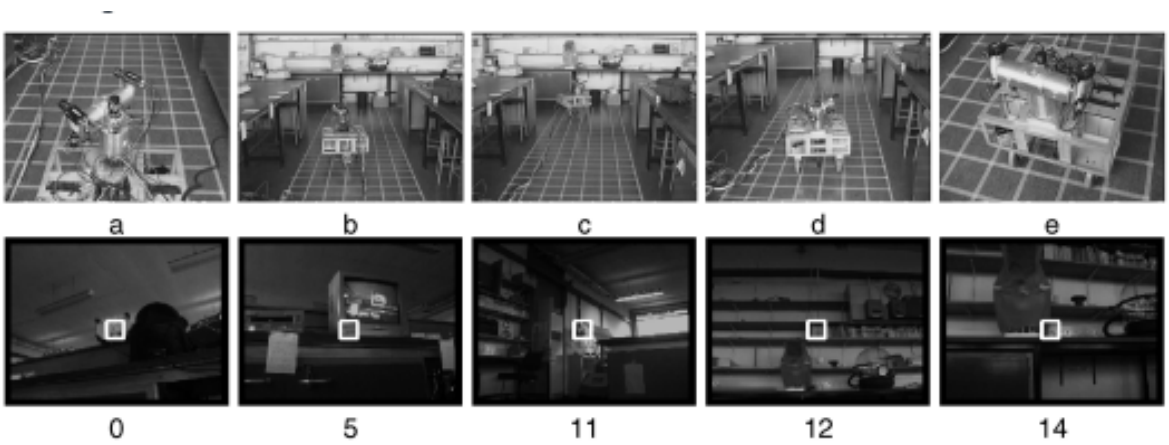



Figure 30: The robot and its active head (top); fixated views of features 0, 5, 11, 12 and 14, each marked by a white square (bottom).

### 6.5.5 Active Localization: Learning Where to Look *(slide 38)*

**Active localization** chooses where the camera should look so that it localizes accurately in a known map, built by structure-from-motion.

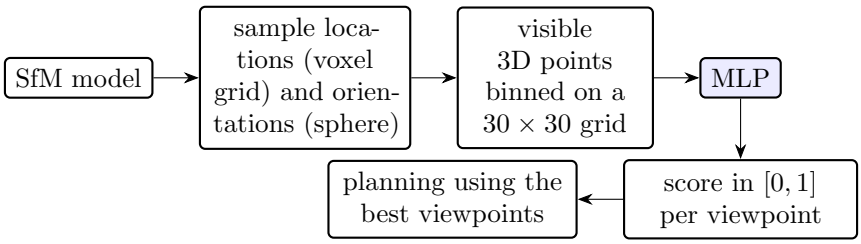



Figure 31: Active localization pipeline (redrawn from slide 38).

- **Candidate viewpoints:** positions on a voxel grid, each with orientations sampled on the sphere; for each, the visible 3D points are binned on a $30 \times 30$ grid and a small MLP scores the viewpoint between 0 and 1.
- **Self-supervised:** each training viewpoint is labelled by registering an image rendered from it against the map; one inference takes 0.02 s.
- 72.9% of 92k test viewpoints localize within 5 cm and 0.4°, against 60.7% for the best choice by Fisher information.

**Intuition**

That is about 67,000 good viewpoints against 56,000, roughly 11,000 more, and 0.02 s per scoring makes planning interactive. The lesson of the slide: *where the points lie in the image* matters more than the Fisher information matrix, which only counts how much information there is, not whether it is spread well enough to constrain pose.

### 6.5.6 Active View Selection with Radiance Fields *(slide 39)*

A **radiance field** is a scene model whose parameters $\boldsymbol{\theta}$ hold radiance and density at points in space, rendered into the image seen from a camera pose by volume rendering. **FisherRF** picks the next camera pose $\mathbf{x}$ by its expected information gain about $\boldsymbol{\theta}$. Let $\mathbf{I}_\mathbf{x} = \mathbf{J}_\mathbf{x}^\top \mathbf{J}_\mathbf{x}$ be the Fisher information of the view ($\mathbf{J}_\mathbf{x}$ the Jacobian of its rendered image with respect to $\boldsymbol{\theta}$; no image from that view is needed) and $\mathbf{I}_\mathcal{D} = \sum_{\mathbf{x}' \in \mathcal{D}} \mathbf{I}_{\mathbf{x}'}$ the sum over the views already captured.

**Proposition 6.5.2: Trace bound on the information gain**

For a candidate view $\mathbf{x}$ with $\mathbf{I}_\mathbf{x} \succeq 0$ and an (invertible, regularized) $\mathbf{I}_\mathcal{D} \succ 0$,

$$\mathrm{IG}(\mathbf{x}) = \tfrac{1}{2} \log\det\!\left(\mathbf{I} + \mathbf{I}_\mathbf{x}\mathbf{I}_\mathcal{D}^{-1}\right) \;\le\; \tfrac{1}{2}\,\mathrm{tr}\!\left(\mathbf{I}_\mathbf{x}\mathbf{I}_\mathcal{D}^{-1}\right).$$

*Proof.* $\mathbf{I}_\mathbf{x}\mathbf{I}_\mathcal{D}^{-1}$ is similar to the PSD matrix $\mathbf{I}_\mathcal{D}^{-1/2}\mathbf{I}_\mathbf{x}\mathbf{I}_\mathcal{D}^{-1/2}$, so it has eigenvalues $\mu_k \ge 0$. Then $\log\det(\mathbf{I} + \cdot) = \sum_k \log(1+\mu_k) \le \sum_k \mu_k = \mathrm{tr}(\cdot)$ because $\log(1+\mu) \le \mu$. □

With over 200 million parameters, both matrices are kept *diagonal*, so the bound becomes $\frac{1}{2}\sum_k (\mathbf{I}_\mathbf{x})_{kk}/(\mathbf{I}_\mathcal{D})_{kk}$. With 3D Gaussian Splatting (a scene of 3D Gaussians), views are evaluated at 70 fps.

**Intuition**

Toy numbers for 3 parameters: $\mathrm{diag}(\mathbf{I}_\mathbf{x}) = (1, 4, 0.1)$ and $\mathrm{diag}(\mathbf{I}_\mathcal{D}) = (10, 1, 0.1)$ give ratios $(0.1, 4, 1)$, bound $\frac{1}{2} \cdot 5.1 = 2.55$. The view is valuable mostly because it informs the second parameter, which previous views barely constrained. In words: *go where the model is least certain and the new image would speak the loudest.*

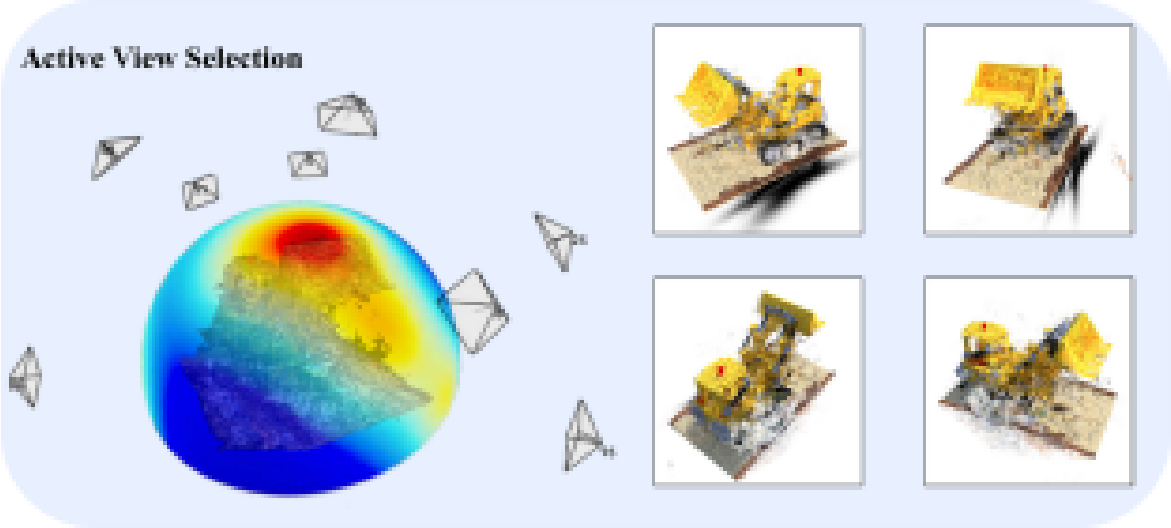



Figure 32: Information gain over the viewing sphere of a scene, with candidate views (frusta) and four of their renders.

### 6.5.7 Outlook: Learned Front Ends *(slide 40)*

Two recent systems keep the back-end machinery of this lecture, least squares over a graph of relative constraints on a Lie group, and replace the feature front end by a network from earlier lectures.

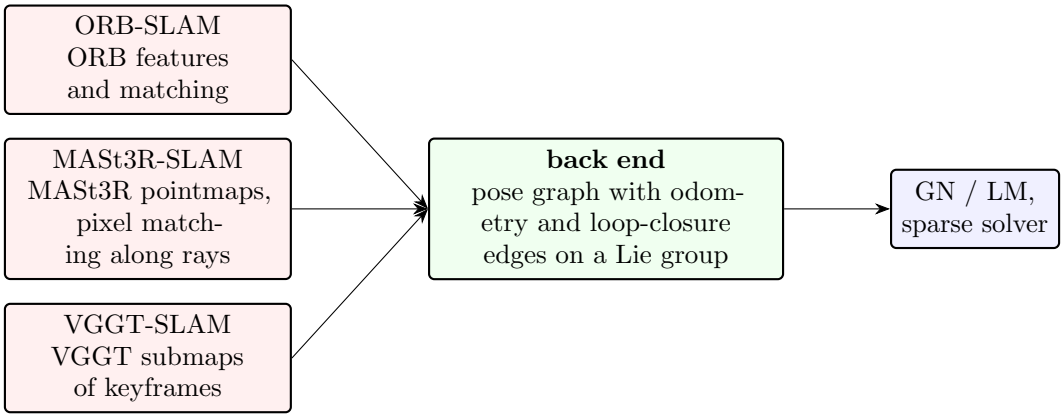
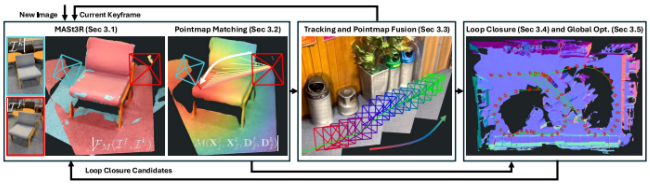



Figure 33: Same back end, different front ends (slide 40).

| | **MASt3R-SLAM** | **VGGT-SLAM** |
|---|---|---|
| Front end | MASt3R (lecture 04) predicts pointmaps of each frame and its keyframe; pixels matched along rays | VGGT (lecture 05) reconstructs a submap of keyframes in one pass |
| Group | keyframe poses in Sim(3) | each submap gets a homography in SL(4) (15 DoF) |
| Solver | Gauss–Newton, sparse Cholesky, on ray errors; loops by image retrieval | pose graph with odometry and loop-closure edges, Levenberg–Marquardt |
| Speed / camera | about 15 FPS on one GPU; no camera model beyond a single centre | without calibration, submaps agree only up to a projective transformation |

**Intuition**

Degrees of freedom of the group tell how much ambiguity the front end leaves: SE(3) has 6 (metric, e.g. stereo), Sim(3) has 7 (one camera: unknown scale), and SL(4) has 15 (an *uncalibrated* reconstruction, where even angles and parallelism are not trustworthy and each submap needs a full projective correction). The back end is the same sparse least squares of Section 6.3; only the group and the residual change.



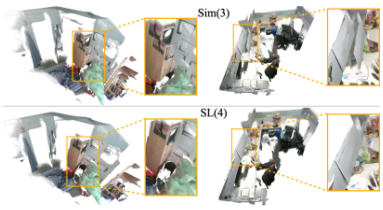




Figure 34: MASt3R-SLAM pipeline (left) and VGGT-SLAM Sim(3) vs. SL(4) comparison renderings (right).

## 6.6 Summary *(slide 41)*

| Method | Estimate / variables | Cost | Loops / drift | Notes |
|---|---|---|---|---|
| EKF-SLAM, MonoSLAM | current pose (13) + $N$ points, one Gaussian | $O(N^2)$ per step and memory; $\approx 100$ features at 30 Hz | no explicit loop mechanism; past poses marginalized | linearized once; may diverge |
| Factor graph / pose graph | all keyframe poses (+ points), MAP = sparse nonlinear LS | GN/LM on sparse $\mathbf{H}$ (Cholesky, CG) | loop edge between non-successive poses | relinearizes; needs good initial guess |
| PTAM | tracking thread + mapping thread, BA on keyframes | BA outside the real-time loop | no large-scale loop closing | small AR workspaces |
| ORB-SLAM | ORB features; 3 threads; local BA; essential graph over Sim(3) | local BA bounded; 21.6 ms/image loop detection | bags of binary words; Sim(3) corrects scale drift | relocalization $\approx 78\%$ of frames |
| Continuous-time splines | control poses of a cumulative B-spline | sliding window of control poses | – | rolling shutter: 4.7% $\rightarrow$ 0.57% error |
| Active SLAM | choose actions by utility (D-opt) | utility per candidate action | revisit vs. explore | active head: largest $3\sigma$ ellipsoid |
| MASt3R-SLAM / VGGT-SLAM | keyframes on Sim(3) / submaps on SL(4) | GN+Cholesky / LM; $\approx 15$ FPS (MASt3R) | image retrieval / pose graph loops | learned front end, same back end |

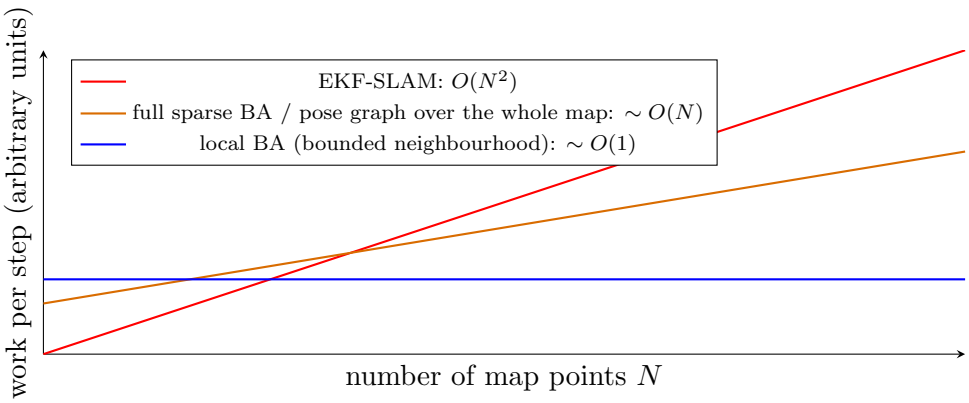



Figure 35: Qualitative scaling of the per-step cost with the map size: the filter pays quadratically for correlations, sparse least squares roughly linearly, and keyframe-based local BA stays bounded (constants are illustrative only).

### Key take-aways

- **SLAM = joint MAP estimation** of trajectory and map from sequential frames; it is SfM with a real-time budget and loop closure.
- **Filtering** (Bayes filter $\rightarrow$ KF $\rightarrow$ EKF-SLAM) keeps one Gaussian over the current pose and map; marginalizing past poses makes the covariance dense, so the cost is $O(N^2)$ and the map must stay tiny (about 100 features at 30 Hz).
- **Graph-based SLAM** keeps the keyframes: MAP on a factor graph is sparse nonlinear least squares, solved by Gauss–Newton/LM; the sparsity pattern of $\mathbf{H}$ equals the graph adjacency.
- **A modern pipeline** (ORB-SLAM) runs tracking (every frame), local mapping (local BA on the covisibility graph) and loop closing (bags of binary words, Sim(3) pose graph to fix scale drift) in parallel.

- **Research directions** change the front end or the problem statement: continuous time for rolling shutter, active viewpoint selection, and learned front ends (MASt3R, VGGT) on top of the same least-squares back end.